Linux System Administration (E1ITA307)

Hackathon-SEE Project Report

On

CLOUD SERVICE PROVIDER

Submitted by

SHOBITH I K (2411021061604)
ALLEN CHRIST A (2411021061266)
SRIKANTH C (2411021061560)
ANASUYA K ANIL (L2411021061702)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Ananthanagu U

ALLIANCE SCHOOL OF ADVANCED COMPUTING
ALLIANCE UNIVERSITY
BENGALURU

Nov 2026



DEPARTMENT OF INFORMATION TECHNOLOGY
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Cloud Service Provider"** submitted by **Shobith I K [2411021061604]**, **Allen Christ A [2411021061266]**, **Srikanth C [2411021061560]** and **Anasuya K Anil [L2411021061702]** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work carried out during the academic year 2026-2027.

Prof. Ananthanagu U	**Prof. Ananthanagu U**	**Dr. Shekhar R**
Course Handling Faculty	Course Coordinator	Professor & HoD

ABSTRACT

In modern enterprise cloud and datacenter computing, shared multi-tenant hosting environments represent a foundational architectural paradigm. Service providers must host heterogeneous client workloads and websites concurrently on unified physical or virtual hardware while guaranteeing uncompromising data confidentiality, robust storage segregation, network containment, and administrative transparency. This project, entitled **Cloud Service Provider** (implemented via the **CloudGuard** platform), designs, deploys, hardens, and validates a multi-customer Linux hosting infrastructure simulating the operational characteristics and security controls of an enterprise cloud hosting provider.

The project is implemented on a dedicated Ubuntu Linux virtual server running within an Oracle VirtualBox hypervisor environment, orchestrated and administered securely from an EndeavourOS Linux workstation. The infrastructure establishes a strict dual-adapter virtual network architecture separating external outbound traffic (NAT) from a private host-only administration and web delivery subnet (192.168.56.0/24). Administrative access is enforced exclusively through OpenSSH public-key cryptography using Ed25519 keypairs, disabling password authentication, keyboard-interactive authentication, and direct root access.

Storage partitioning and volume allocation are built entirely on a dedicated 25 GiB secondary virtual disk (/dev/sdb) formatted with a GUID Partition Table (GPT) and orchestrated via Logical Volume Manager (LVM2). Within the unified volume group *cloudguard_vg*, three distinct ext4 logical volumes are carved: *client1_lv* (8 GiB mounted at /srv/client1), *client2_lv* (8 GiB mounted at /srv/client2), and *backup_lv* (4 GiB mounted at /srv/backups), reserving unallocated storage pool capacity for dynamic scaling. Customer accounts *client1* and *client2* are isolated at the POSIX permissions layer using mode 2750 (setgid) and dedicated ownership, ensuring strict cross-tenant directory traversal denial while enabling the Nginx web server worker process (group *www-data*) to serve customer web content across independent virtual hosts (client1.cloudguard.local and client2.cloudguard.local).

Infrastructure resilience and security are reinforced by Uncomplicated Firewall (UFW) stateful ingress filtering restricted to the host-only interface, an automated Bash strict-mode backup system scheduled via cron with a 7-generation retention policy, an hourly system health monitor, and an end-to-end disaster recovery drill. Finally, comprehensive administrative observability is achieved through the **CloudGuard Administration Dashboard**, a custom read-only Flask and Gunicorn web service protected by HTTP Basic Authentication and isolated via systemd sandboxing directives. The project demonstrates that standard open-source Linux utilities can be systematically engineered into a resilient, secure, and observable multi-tenant cloud service platform.

Keywords: Linux System Administration, Multi-Tenancy, Cloud Service Provider, LVM2, OpenSSH Hardening, Nginx Virtual Hosting, UFW Firewall, POSIX Permissions, Backup Retention, Flask, systemd Hardening.

Table of Contents

1. Introduction **4**
1.1 Aim 4
1.2 Background and Theoretical Foundations 4
1.3 Problem Statement 5
1.4 Project Objectives 5
1.5 Scope and Operational Boundaries 6
2. Methodology **7**
2.1 Tools, Technologies, and Environment Requirements 7
2.1.1 Software Requirements 7
2.1.2 Hardware and Virtualization Requirements 8
2.1.3 Administrative and Diagnostic Tooling 9
2.2 Chronological Development Process and Workflow 10
3. Linux System Administration – Design and Implementation **12**
3.1 System Architecture and Multi-Tenant Principles 12
3.2 Linux Environment and Virtual Machine Setup 15
3.2.1 Operating System and Hypervisor Plan 15
3.2.2 Package Ecosystem and Repository Synchronization 17
3.2.3 Remote Administration and OpenSSH Daemon Architecture 18
3.2.4 Dual Virtual Network Interface Configuration 20
3.3 Customer Identity and Account Management 22
3.3.1 Customer Linux Account Provisioning 22
3.3.2 Credentials Management and Authentication Policies 23
3.3.3 Numeric UIDs, GIDs, and Security Implications 24
3.3.4 Web Server Integration Group Mechanics 25
3.4 File and Directory Management 26
3.4.1 Directory Hierarchy and Layout Design 26
3.4.2 Ownership and Mode 2750 setgid Enforcement 27
3.4.3 Customer Website Deployment and Assets 29
3.4.4 Directory Isolation Verification Procedures 30
3.5 Package and Service Process Management 31
3.5.1 Package Lifecycle and Maintenance 31
3.5.2 Tool Verification and Binary Environment 32
3.5.3 Systemd Service Management and States 33
3.5.4 Resource and Process Monitoring 34
3.6 Storage and File System Configuration 35
3.6.1 Storage Fundamentals and Multi-Tenant Isolation 35
3.6.2 Identifying and Inspecting the Dedicated Data Disk 37

3.6.3 GUID Partition Table (GPT) Creation 38
3.6.4 Validating Partition Boundaries and Alignment 39
3.6.5 Logical Volume Management (LVM2) Architecture 40
3.6.6 Creating ext4 File Systems and Inode Structures 43
3.6.7 Mounting Logical Volumes 44
3.6.8 Persistent Mounts with /etc/fstab and UUIDs 45
3.6.9 Storage Allocation and Capacity Mapping Summary 46
3.7 Nginx Virtual Hosting and Web Service Deployment 47
3.7.1 Web Server Architecture and Virtual Host Mechanics 47
3.7.2 Virtual Host Server Block Definitions 48
3.7.3 Default Virtual Host Deactivation and Syntax Verification 50
3.7.4 Local Hostname Resolution and Routing 51
3.7.5 Web Delivery Validation and HTTP Header Analysis 52
3.8 Automated Backup System and Disaster Recovery 53
3.8.1 Multi-Tenant Backup Architecture and Requirements 53
3.8.2 Shell Script Engineering and Bash Strict Mode 54
3.8.3 Automated 7-Generation Retention Algorithm 56
3.8.4 Cron Scheduling and Execution Environment 57
3.8.5 Backup Logging and Audit Tracing 58
3.8.6 Disaster Recovery and File Restoration Drill 59
3.9 System Monitoring and Health Automation 61
3.9.1 Monitoring Objectives and Health Metrics 61
3.9.2 Shell Script Architecture and Threshold Logic 62
3.9.3 Cron Hourly Scheduling and Execution Tracing 63
3.9.4 System Health Audit Log Analysis 64
3.10 Security Hardening and Access Control 65
3.10.1 Defense-in-Depth Multi-Layered Security Model 65
3.10.2 UFW Firewall Configuration and Subnet Restriction 66
3.10.3 OpenSSH Cryptographic Hardening 68
3.10.4 Filesystem Least-Privilege Enforcement 70
3.10.5 Troubleshooting Common Administrative Failures 71
3.11 CloudGuard Administration Dashboard 73
3.11.1 Purpose, Design Philosophy, and Observability 73
3.11.2 Architecture and Technology Stack 74
3.11.3 REST API Endpoints and Unprivileged Data Aggregation 75
3.11.4 HTTP Basic Authentication and Credential Protection 77
3.11.5 Dedicated Service Account Isolation 78
3.11.6 Systemd Sandboxing and Hardening Directives 79
3.11.7 Network Binding and Gunicorn WSGI Deployment 81
4. Results and Testing **82**

4.1 Implementation Evidence and Technical Results 82
4.1.1 Overview of Evidence Inventory and Screenshots 82
4.1.2 Storage Configuration Evidence (Figure 4.1) 83
4.1.3 Customer Account Isolation Evidence (Figure 4.2) 85
4.1.4 Web Server Hosting Evidence (Figure 4.3) 87
4.1.5 Network Security and SSH Hardening Evidence (Figure 4.4) 89
4.1.6 Automated Backup System Evidence (Figure 4.5) 91
4.1.7 Monitoring and Scheduling Evidence (Figure 4.6) 93
4.1.8 Dashboard Service Hardening Evidence (Figure 4.7) 95
4.1.9 Comprehensive CloudGuard Acceptance Checklist 97
4.2 Functional Validation and Recovery Testing 100
4.2.1 Comprehensive Functional Test Matrix (T01–T11) 100
4.2.2 Disaster Recovery and File Restoration Drill 103
4.2.3 Data Integrity and Security Boundary Validation 105
4.3 Dashboard Results and Operational Testing 106
4.3.1 Visual Interface Execution (Figures 4.8 & 4.9) 106
4.3.2 REST API Verification with curl 109
4.3.3 Negative Security and Authentication Testing 111
4.3.4 Comprehensive Dashboard Test Cases Matrix 112
5. Conclusion and Future Scope 114
5.1 Project Conclusion 114
5.2 Technical Limitations 115
5.3 Future Scope and Strategic Enhancements 117
References 119
Appendices 121
Appendix A: Detailed Architecture and Network Workflow Diagrams 121
Appendix B: Linux System Administration Commands Quick Reference 126
Appendix C: Complete Production Scripts and System Configurations 133
Appendix D: CloudGuard Administration Dashboard — Full Source Code Listing 141
Appendix E: System Configuration Manifests, Audit Records, and Evidence Logs 153

1. Introduction

1.1 Aim

The primary aim of this project is to architect, configure, harden, and empirically validate a comprehensive Linux-based Cloud Service Provider hosting platform, designated as **CloudGuard**. The project demonstrates how standard open-source Linux system administration tools, system services, and security abstractions can be synthesized to deliver a robust multi-tenant hosting environment on a single virtualized Linux server. The platform simulates core infrastructure offerings of modern cloud service providers—specifically multi-tenant storage provisioning, strong customer process and filesystem isolation, independent name-based virtual web hosting, perimeter and transport security, automated disaster recovery backups with strict retention policies, continuous health monitoring, and unified administrative observability via a hardened web dashboard.

Rather than relying on proprietary cloud orchestration frameworks or opaque automation layers, the project builds the entire service provider architecture from foundational Linux principles. Beginning with raw disk partitioning using the GUID Partition Table (GPT), the storage subsystem is structured using the Logical Volume Manager (LVM2) to dynamically allocate ext4 filesystems across segregated customer volumes and backup pools. Customer web workloads are hosted using Nginx virtual servers configured with POSIX directory permissions that enforce cross-tenant confidentiality while permitting web service traversal. Inbound network perimeters are restricted using the Uncomplicated Firewall (UFW), remote management is secured via cryptographic OpenSSH hardening, and routine operational workflows are automated using cron. The completed system provides a transparent, verifiable, and fully reproducible reference model for secure multi-tenant Linux server administration.

1.2 Background and Theoretical Foundations

In the contemporary landscape of enterprise information technology, cloud computing infrastructure represents the cornerstone of software deployment and web service delivery. Organizations of all scales routinely rely on cloud service providers to host mission-critical web applications, application programming interface (API) gateways, and enterprise data stores. At the foundational hardware and operating system tier of virtually all public and private clouds—including Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure—lies the Linux operating system kernel. Linux's open-source architecture, granular POSIX permissions model, modular storage management capabilities, and robust network protocol stack make it the premier choice for infrastructure virtualization and multi-tenant hosting.

A central engineering imperative of any shared hosting or cloud provider platform is **multi-tenancy**: the architectural ability to serve multiple distinct clients (tenants) from a consolidated physical or virtual server instance. In a multi-tenant environment, diverse customers share identical underlying computational resources—such as the central processing unit (CPU), random-access memory (RAM),

disk controllers, and physical network adapters. However, from the perspective of each tenant, their storage volumes, application files, and web presences must appear completely private and isolated. Inadvertent cross-tenant data leakage constitutes an unacceptable failure mode that violates regulatory compliance (e.g., GDPR, HIPAA, PCI-DSS) and destroys client trust.

Under the standard POSIX security model implemented in Linux, identity is fundamentally numeric. The operating system kernel does not track access requests using textual usernames or group names; rather, all discretionary access control (DAC) checks are evaluated against numeric User Identifiers (UIDs) and Group Identifiers (GIDs). Each file and directory on an ext4 filesystem maintains an inode containing ownership metadata (UID and GID) alongside a 12-bit permission mode mask (comprising read, write, execute permissions for owner, group, and others, as well as setuid, setgid, and sticky special bits). Achieving secure multi-tenancy demands rigorous alignment between account identities, group memberships, and directory permission masks. For example, if a shared web server daemon (such as Nginx) requires read access to a customer's document root, granting that access must not simultaneously expose the customer's directory to other unprivileged tenant accounts on the same system.

Furthermore, modern cloud infrastructure mandates dynamic storage management. Traditional static disk partitioning schemes (such as the legacy Master Boot Record format) impose rigid volume size limits and cannot be resized or reorganized without taking the filesystem offline and risking data corruption. The Logical Volume Manager (LVM2) abstracts physical storage devices into flexible storage pools. By aggregating Physical Volumes (PVs) into a Volume Group (VG), administrators can carve out Logical Volumes (LVs) that can be individually resized, formatted with distinct filesystems, mounted to designated paths, and backed up independently. In a service provider context, LVM ensures that one tenant cannot exhaust the disk space of another tenant, establishing critical operational boundaries.

1.3 Problem Statement

The problem statement addressed by this project is defined as follows:
“Design, implement, harden, and document a secure multi-customer Linux Cloud Service Provider platform (CloudGuard) on an Ubuntu virtual server, featuring LVM storage partitioning, customer account and filesystem isolation, Nginx name-based virtual hosting, UFW firewall controls, OpenSSH cryptographic hardening, automated backup retention, disaster recovery validation, and a centralized web administration dashboard.”

Operating a multi-tenant hosting server presents significant technical challenges and operational risks when managed without rigorous administrative discipline. A poorly configured hosting server is susceptible to numerous critical vulnerabilities:

- **Cross-Tenant Directory Traversal:** Overly permissive directory permissions (e.g., world-readable modes such as 755 or 777) allow malicious or compromised customer accounts to inspect, exfiltrate, or tamper with neighboring clients' sensitive proprietary code, database

credentials, and website assets.

- **Shared Storage Contention and Disk Exhaustion:** If all customer data resides within an unconstrained root filesystem (/), a single runaway process, log explosion, or malicious storage abuse by one client can consume 100% of available disk space, precipitating kernel panics and taking down all hosted clients simultaneously.
- **Perimeter Exposure and Brute-Force Attacks:** Leaving administrative management interfaces (such as SSH) exposed to unauthenticated password attacks or direct root login allows malicious network actors to compromise full superuser control via dictionary or automated credential-stuffing exploits.
- **Catastrophic Data Loss and Backup Failures:** Without automated, versioned, and retention-managed backups, hardware errors, accidental deletions, or filesystem corruptions lead to irreversible data loss. Furthermore, unverified backups provide a false sense of security unless tested via formal disaster recovery restoration drills.
- **Administrative Blind Spots:** Relying exclusively on manual CLI diagnostics causes delayed incident detection. System administrators lack an immediate, real-time synthetic view of system health, storage saturation, service availability, and tenant status.

This project systematically resolves each of these vulnerabilities by engineering an integrated, automated, and observable Linux cloud hosting infrastructure that demonstrates best practices in enterprise system administration.

1.4 Project Objectives

To satisfy the project requirements, the following specific engineering objectives were defined and executed:

1. **Virtual Server and Network Provisioning:** Deploy an Ubuntu 26.04.1 LTS virtual server within Oracle VirtualBox, configured with dual virtual network adapters separating external outbound traffic (NAT) from a private, controlled host-only administrative and web delivery subnet (192.168.56.0/24).
2. **Cryptographic Remote Administration:** Establish a hardened OpenSSH administrative bridge between the EndeavourOS host workstation and the Ubuntu server using Ed25519 public-key authentication, disabling password login and direct root access.
3. **Dynamic LVM Storage Architecture:** Attach a dedicated 25 GiB secondary virtual disk (/dev/sdb), format it with a GPT partition table, initialize LVM2 physical volumes and volume groups (cloudguard_vg), and carve out dedicated ext4 logical volumes for Client 1 (8 GiB), Client 2 (8 GiB), and Backups (4 GiB), ensuring persistent mounting via filesystem UUIDs in /etc/fstab.
4. **Multi-Tenant Account and Permission Isolation:** Create distinct Linux user accounts (client1 and client2) and enforce POSIX directory permissions (mode 2750 with setgid) that prevent cross-customer access while enabling the Nginx worker group (www-data) to access public web

content.

5. **High-Performance Web Hosting:** Deploy and configure Nginx virtual hosts to deliver independent static websites for both customers using name-based HTTP virtual hosting (client1.cloudguard.local and client2.cloudguard.local).
6. **Network Perimeter Hardening:** Configure Uncomplicated Firewall (UFW) with a default-deny incoming policy, restricting inbound traffic strictly to SSH (port 22), HTTP (port 80), and Dashboard (port 5000) on the private host-only interface.
7. **Automated Backup and Retention System:** Author a robust Bash strict-mode backup script utilizing rsync to generate timestamped archives, enforce an automated 7-snapshot retention policy, and schedule daily execution via cron.
8. **Disaster Recovery Validation:** Conduct an empirical disaster recovery test by creating, backing up, deleting, and successfully restoring a designated test file, verifying data integrity.
9. **System Monitoring and Cron Scheduling:** Implement an automated health monitoring script that samples CPU, RAM, disk, LVM, and service availability, triggering warning logs when resources exceed 80% capacity.
10. **Administration Dashboard Development:** Build, harden, and deploy the CloudGuard Administration Dashboard using Python Flask, Gunicorn, and systemd sandboxing directives, providing secure, read-only operational observability.

1.5 Scope and Operational Boundaries

In accordance with academic rigor and the master implementation documentation, the operational scope of this project is precisely bounded. It is essential to delineate what was constructed versus what lies outside the implemented scope of this laboratory-grade demonstration.

Implemented Scope:

- Single-node Ubuntu virtual server running in VirtualBox on an x86_64 hardware platform.
- Two fully configured customer tenants: client1 and client2 with isolated home and web trees.
- Secondary 25 GiB virtual disk partitioned with GPT and managed via LVM2.
- Two independent static customer websites served via Nginx virtual host server blocks on port 80.
- Private VirtualBox host-only network (192.168.56.0/24) with local /etc/hosts resolution on the management workstation.
- Automated rsync-based backups with strict 7-run retention and cron automation.
- Read-only web administration dashboard running on port 5000 via Gunicorn and systemd.

Explicitly Out of Scope (Academic Boundaries):

- Public Internet routing, commercial cloud deployments (AWS, GCP), or public DNS registrar propagation.

- TLS/HTTPS certificates; all demonstrated HTTP traffic operates over port 80 within the private host-only subnet.
- Dynamic relational databases (MySQL/PostgreSQL), customer billing engines, or automated self-service registration portals.
- Multi-node clustering, container orchestration (Kubernetes/Docker), or automated hardware failover.
- Off-site remote backup replication; backup archives are held on a dedicated logical volume on the same host.

2. Methodology

2.1 Tools, Technologies, and Environment Requirements

The implementation of the CloudGuard Cloud Service Provider platform requires a cohesive synthesis of Linux operating system components, virtualization technologies, cryptographic protocols, storage abstraction layers, and web hosting services. The development environment was architected to emulate a real-world enterprise cloud datacenter scenario while operating within a controlled, observable laboratory hypervisor.

2.1.1 Software Requirements

The project relies entirely on production-grade, open-source Linux software utilities and daemons. Table 2.1 outlines the primary software packages, their functional roles within the CloudGuard ecosystem, and their operational relevance.

Table 2.1: Software requirements and functional roles

Software / Tool	Package / Version	Operational Role in CloudGuard
Ubuntu Linux	26.04.1 LTS (Guest)	Base operating system for the CloudGuard server VM, providing Linux kernel 7.0.0-43-generic and systemd.
EndeavourOS	Rolling (Host)	Host Linux workstation providing the hypervisor host, administration terminal, and web browser client.
Oracle VirtualBox	7.x Hypervisor	Type-2 hypervisor hosting the server VM, providing virtual hardware, snapshots, and dual network adapters.
OpenSSH Server	openssh-server	Secure remote administration daemon supporting Ed25519 public-key authentication and cryptographic hardening.
LVM2 Utilities	lvm2 (pv/vg/lv)	Logical Volume Manager for flexible partitioning, volume group creation, and dynamic logical volume provisioning.
e2fsprogs	mkfs.ext4, fsck	Utilities for formatting, mounting, and verifying ext4 journaling filesystems with POSIX attributes.
Nginx Web Server	nginx (core)	High-performance reverse proxy and web server providing name-based virtual hosting for client websites on port 80.
UFW Firewall	ufw / iptables	Host-level stateful firewall managing network packet filtering and ingress port restrictions on private interfaces.
rsync & tar	rsync, tar	High-speed file synchronization and archival utilities used by the automated backup system with metadata preservation.
cron (cronie)	cron daemon	System task scheduler managing periodic execution of automated backup scripts and system health monitors.
Python 3 & venv	python3, python3-venv	Runtime environment and isolated virtual environment for the CloudGuard Administration Dashboard.
Flask Framework	flask >= 3.1	Micro-web framework providing REST API endpoints (/api/overview, /health) and serving dashboard templates.
Gunicorn WSGI	gunicorn >= 23	Production-grade WSGI HTTP server executing Flask application workers behind a systemd service wrapper.
psutil Library	psutil >= 7	Cross-platform process and system monitoring library gathering real-time CPU, RAM, disk, and uptime metrics.
Diagnostics	curl, htop, net-tools	Command-line diagnostic and inspection tools used for system profiling, verification, and API validation.

2.1.2 Hardware and Virtualization Requirements

The project was engineered and hosted on a physical laptop workstation featuring x86_64 hardware virtualization (Intel VT-x / AMD-V) enabled within the UEFI firmware. Within the Oracle VirtualBox hypervisor, a dedicated virtual machine named CloudGuard-Server was provisioned with the following hardware specifications:

- **Host System:** Physical laptop running EndeavourOS Linux (KDE Plasma desktop environment) acting as the administrative workstation, SSH client, and testing origin.
- **Virtual CPU Allocation:** 2 vCPUs assigned to the guest VM to support concurrent execution of Nginx worker processes, Gunicorn WSGI threads, and background cron scripts.
- **System Memory (RAM):** 4096 MiB (4 GiB) allocated to the virtual machine, with approximately 3.3 GiB visible and available to user-space processes.
- **Operating System Virtual Disk (/dev/sda):** 25 GB dynamically allocated VDI virtual disk housing the base Ubuntu root filesystem (/), bootloader, and core binaries.
- **Dedicated Data Virtual Disk (/dev/sdb):** 25 GiB secondary VDI virtual disk attached to the SATA controller, dedicated entirely to customer data volumes and backup archives.
- **Network Adapter 1 (NAT):** VirtualBox Network Address Translation adapter attached to enp0s3 (10.0.2.15/24) for secure outbound package installation and security updates.
- **Network Adapter 2 (Host-Only):** VirtualBox Host-Only Ethernet Adapter attached to enp0s8 (192.168.56.103/24) interfacing with host interface vboxnet0 for private administration and website traffic.

2.1.3 Administrative and Diagnostic Tooling

A rigorous set of system administration utilities was leveraged to ensure complete observability, configuration accuracy, and repeatable testing throughout the implementation lifecycle:

- **Hypervisor Snapshot Mechanism:** Prior to executing critical configuration milestones, a base snapshot named Ubuntu-Base-SSH-Ready was generated in VirtualBox. This provided an immutable rollback state, guaranteeing that unexpected package conflicts or filesystem errors could be reverted without restarting VM provisioning.
- **Cryptographic Key Management:** OpenSSH ssh-keygen was employed on the EndeavourOS host to generate a 256-bit Ed25519 elliptic-curve keypair (~/.ssh/cloudguard_ed25519), delivering superior security and performance over legacy RSA keys.
- **Host Configuration Aliasing:** The host OpenSSH configuration (~/.ssh/config) was populated with a dedicated cloudguard alias, encapsulating connection parameters, port numbers, usernames, and key associations for streamlined administrative sessions.
- **Terminal Multiplexing and Logging:** Terminal sessions were managed with interactive diagnostic logging, verifying return codes (echo $?), inspecting service unit states via systemctl, and capturing pristine terminal evidence.

2.2 Chronological Development Process and Workflow

The project methodology was structured into ten sequential, interdependent engineering phases. This chronological workflow ensured that foundational layers (hypervisor, networking, storage) were fully verified and stabilized prior to deploying application-tier services (Nginx, Gunicorn) and security hardening layers.

Phase 1: Environment Provisioning and Base Setup: The Ubuntu 26.04.1 LTS virtual server was initialized in VirtualBox with dual network adapters (NAT on enp0s3 and Host-Only on enp0s8). The administrative user account cloudadmin was created with sudo privileges, and system timezone was synchronized to Asia/Kolkata.

Phase 2: Remote Administration and SSH Key Architecture: OpenSSH was configured on the server. On the EndeavourOS host, an Ed25519 keypair was generated and authorized on the guest VM. An SSH alias was established in ~/.ssh/config. Initial key path misconfigurations were diagnosed and resolved, validating passwordless administrative login.

Phase 3: Package Ecosystem and Toolchain Installation: The Ubuntu APT repository was synchronized, and core server packages were installed—including lvm2, nginx, ufw, rsync, python3-venv, and diagnostic utilities. Daemon states were verified via systemctl.

Phase 4: Dedicated Storage and LVM Partitioning: The 25 GiB secondary disk (/dev/sdb) was partitioned using GPT to create /dev/sdb1 (type Linux LVM). An LVM physical volume and volume group (cloudguard_vg) were initialized. Three logical volumes (client1_lv, client2_lv, backup_lv) were carved, formatted with ext4, and mounted persistently via UUID entries in /etc/fstab.

Phase 5: Customer Account Creation and Permission Isolation: Dedicated Linux user accounts (client1 and client2) were provisioned with disabled interactive shells. Directory hierarchies were created under /srv/client1 and /srv/client2. POSIX permissions were locked to mode 2750 (setgid) with ownership assigned to client:www-data, enforcing strict cross-tenant directory traversal denial.

Phase 6: Nginx Virtual Web Hosting Deployment: Nginx was configured with separate virtual host server blocks in /etc/nginx/sites-available/ for client1.cloudguard.local and client2.cloudguard.local. The default site configuration was deactivated. Customer landing pages were deployed, and host-side /etc/hosts mappings were created to enable name-based web testing.

Phase 7: Network Perimeter and SSH Cryptographic Hardening: UFW was enabled with a default-deny incoming policy, permitting traffic on ports 22, 80, and 5000 strictly from the 192.168.56.0/24 subnet on enp0s8. OpenSSH was hardened via a drop-in configuration (/etc/ssh/sshd_config.d/99-cloudguard.conf) that disabled password authentication, root login, and keyboard-interactive auth.

Phase 8: Automated Backup Engineering and Retention Logic: A production-grade Bash backup script (/usr/local/sbin/cloudguard-backup.sh) was authored using strict mode (set -Eeuo pipefail). The script uses rsync to capture timestamped archives to /srv/backups/cloudguard, prunes archives

exceeding a 7-snapshot retention limit, logs executions to /var/log/cloudguard-backup.log, and runs daily at 02:00 via cron.

Phase 9: Disaster Recovery Drill and System Health Monitoring: A disaster recovery test was executed by creating a test file in customer storage, capturing a backup, deleting the live file, and successfully restoring it from archive. Additionally, an hourly monitoring script (/usr/local/sbin/cloudguard-monitor.sh) was deployed to track CPU, memory, disk, LVM, and service states with 80% warning thresholds.

Phase 10: Administration Dashboard Development and Service Sandboxing: A lightweight, read-only administrative web dashboard was built using Flask, Gunicorn, and psutil. The service was sandboxed under a non-login user (cloudguard-dashboard) and secured using systemd hardening directives (NoNewPrivileges, ProtectSystem=strict, PrivateTmp). HTTP Basic Authentication was integrated, and end-to-end operational verification was completed.

3. Linux System Administration – Design and Implementation

3.1 System Architecture and Multi-Tenant Principles

The architecture of the **CloudGuard** Cloud Service Provider platform is designed around strict principles of physical and logical resource segregation, defense-in-depth security, and service modularity. The platform is structured across two primary computing boundaries: the physical **EndeavourOS Management Workstation** (the administrative and client terminal) and the virtualized **CloudGuard-Server Virtual Machine** (the multi-tenant hosting engine running Ubuntu 26.04.1 LTS). Rather than executing workloads directly on the host or mixing administrative tasks with customer workloads, all multi-tenant operations are encapsulated within the virtual server.

Multi-tenancy on Linux demands that three critical planes be separated: the *Storage Plane*, the *Network Plane*, and the *Application/Service Plane*. In CloudGuard, the storage plane isolates tenant file trees onto dedicated LVM logical volumes residing on a secondary virtual drive. The network plane segregates administrative and web ingress traffic onto a private host-only subnet (192.168.56.0/24), strictly isolated from public or bridged physical networks. The application plane ensures that Nginx server blocks and the administrative dashboard run under unprivileged, sandboxed identities.

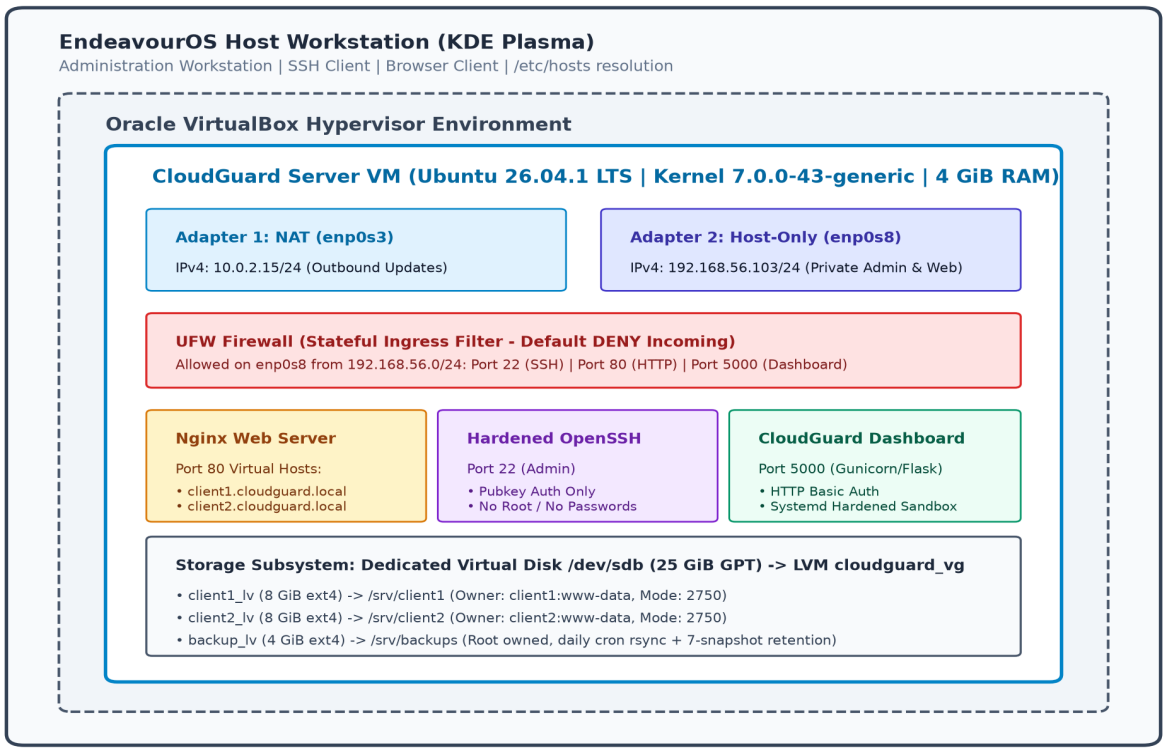


Figure 3.1: CloudGuard system architecture and multi-tenant hosting tiers

Primary Architectural Components and Tiers:

- **1. Management Workstation (EndeavourOS):** The physical development and administrative laptop running EndeavourOS Linux with KDE Plasma. It serves as the authoritative SSH client terminal, repository origin, local DNS resolver (via /etc/hosts), and web testing client.
- **2. Virtualization Layer (Oracle VirtualBox 7.x):** The type-2 hypervisor that abstracts physical CPU, RAM, and storage into isolated virtual hardware. It manages dual virtual network interfaces and provides virtual disk controllers for independent OS and data drives.
- **3. Guest Operating System (Ubuntu 26.04.1 LTS):** The server operating system running Linux kernel 7.0.0-43-generic on x86_64 architecture. It provides the kernel namespace, POSIX user management, systemd service supervision, and network stack.
- **4. Storage Subsystem (LVM2 on /dev/sdb):** A dedicated 25 GiB secondary disk managed via Logical Volume Manager. It carves the raw disk into volume group cloudguard_vg, hosting independent ext4 logical volumes for Client 1, Client 2, and the Backup vault.
- **5. Ingress Security & Firewall (UFW):** Netfilter/iptables wrapper enforcing stateful packet inspection. It implements a default-deny policy for incoming traffic, restricting port access strictly to host-only subnet 192.168.56.0/24.
- **6. Web Server Engine (Nginx):** High-concurrency HTTP server listening on TCP port 80. It utilizes name-based virtual hosting to route incoming HTTP Host headers to the appropriate customer document root.
- **7. Automation & Backup Engine (cron & rsync):** Autonomous background subsystem executing daily strict-mode rsync backups, managing a 7-snapshot retention cycle, and sampling hourly health metrics.
- **8. Observability & Dashboard (Flask / Gunicorn / systemd):** Administrative web interface on port 5000 providing real-time visibility into CPU, memory, storage mounts, service health, customer status, and backup logs.

Table 3.1: CloudGuard system architecture parameters and specifications

Subsystem / Tier	Design Specification	Technical Parameters
Host Workstation	EndeavourOS Linux (Rolling)	Kernel 6.x+, x86_64, KDE Plasma, OpenSSH client, curl, Firefox
Hypervisor	Oracle VirtualBox 7.x	Type-2 Hypervisor, hardware virtualization VT-x enabled, vboxnet0 host-only
Guest VM Name	CloudGuard-Server	Ubuntu 26.04.1 LTS, Hostname: cloudguard-server, Timezone: Asia/Kolkata
Virtual Hardware	2 vCPUs, 4 GiB Memory	3.3 GiB RAM visible, ACPI enabled, SATA AHCI storage controller
Root Storage Disk	/dev/sda (25 GB VDI)	Base Ubuntu installation, root filesystem (/), bootloader (GRUB)
Data Storage Disk	/dev/sdb (25 GiB VDI)	Dedicated data disk, GPT partition /dev/sdb1, LVM2 volume group
Network Adapter 1	NAT (enp0s3)	IPv4 10.0.2.15/24, gateway 10.0.2.2, outbound Internet access only
Network Adapter 2	Host-Only (enp0s8)	IPv4 192.168.56.103/24, subnet 192.168.56.0/24, private admin & web
SSH Administration	OpenSSH 9.x+ (TCP 22)	Ed25519 public key auth, password auth disabled, root login disabled
Web Delivery	Nginx (TCP 80)	Name-based vhosts: client1.cloudguard.local, client2.cloudguard.local
Dashboard Service	Flask + Gunicorn (TCP 5000)	Bound to 192.168.56.103:5000, HTTP Basic Auth, systemd sandboxing
Backup Strategy	rsync + cron (Daily 02:00)	Timestamped directories under /srv/backups/cloudguard, 7-generation retention

3.2 Linux Environment Setup

3.2.1 Operating System and Hypervisor Plan

The selection of Ubuntu 26.04.1 LTS as the guest operating system provides a stable, enterprise-aligned foundation featuring long-term software support, modern kernel abstractions (Linux 7.0.0-43-generic), robust systemd service supervision, and widespread package availability via Canonical's Advanced Package Tool (APT) repositories. VirtualBox was configured with an ACPI-compliant motherboard profile and an AHCI SATA controller supporting two distinct virtual drives.

To establish configuration safety, a strict snapshotting regimen was enforced. Immediately following base OS installation, administrative account provisioning (cloudadmin), and initial SSH configuration, a comprehensive snapshot entitled Ubuntu-Base-SSH-Ready was captured via the VirtualBox

management console. This snapshot recorded the exact block-level state of both virtual disks and VM memory, providing an instant recovery baseline should subsequent package installations, LVM volume creations, or firewall policies compromise network connectivity.

```
# Verifying system identity and kernel parameters on CloudGuard VM
uname -a
# Output: Linux cloudguard-server 7.0.0-43-generic #43-Ubuntu SMP PREEMPT_DYNAMIC x86_64
timedatectl
# Output: Time zone: Asia/Kolkata (IST, +0530) | System clock synchronized: yes
hostnamectl status
# Output: Operating System: Ubuntu 26.04.1 LTS | Architecture: x86-64
```

3.2.2 Package Ecosystem and Repository Synchronization

Prior to compiling application services or configuring storage pools, the guest package database was synchronized with Canonical's official upstream mirrors over the NAT network interface (enp0s3). Core system administration, cryptographic, web, and storage packages were installed using the non-interactive APT wrapper.

```
# Synchronizing package indices and upgrading installed components
sudo apt-get update -y && sudo apt-get upgrade -y

# Installing primary infrastructure and diagnostic packages
sudo apt-get install -y \
    lvm2 e2fsprogs parted gdisk \
    nginx ufw openssh-server \
    rsync tar cron curl wget git \
    python3 python3-pip python3-venv \
    net-tools dnsutils htop vim nano
```

3.2.3 Remote Administration and OpenSSH Daemon Architecture

Remote administration of cloud infrastructure must be completely decoupled from unauthenticated or password-dependent vectors. To administer the CloudGuard virtual server securely from the EndeavourOS host, a 256-bit Ed25519 elliptic-curve keypair was generated on the host workstation. Ed25519 was chosen over traditional RSA keys due to its superior resistance to side-channel timing attacks, significantly smaller key footprint (68 characters vs. 2048+ bits), and rapid cryptographic signing.

```
# On EndeavourOS Management Host: Generating dedicated Ed25519 administrative keypair
ssh-keygen -t ed25519 -a 100 -f ~/.ssh/cloudguard_ed25519 -C 'shobith@cloudguard-admin'

# Deploying public key to Ubuntu CloudGuard VM cloudadmin authorized_keys
ssh-copy-id -i ~/.ssh/cloudguard_ed25519.pub cloudadmin@192.168.56.103
```

To streamline administrative sessions and prevent manual identity errors, an SSH client configuration alias was created in ~/.ssh/config on the EndeavourOS host. This profile binds the alias cloudguard to the target host-only IP address, specifies the non-root administrative account cloudadmin, enforces the

Ed25519 identity key, and restricts authentication attempts exclusively to the specified key.

```
# Host Configuration: ~/.ssh/config entry for CloudGuard administration
Host cloudguard
    HostName 192.168.56.103
    User cloudadmin
    IdentityFile ~/.ssh/cloudguard_ed25519
    IdentitiesOnly yes
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

Diagnostic Troubleshooting Encountered During Setup: During initial testing, an administrative connection attempt failed with Identity file ~/.ssh/cloudguard_ed25519 not accessible: No such file or directory. Investigation revealed that the connection command was accidentally executed from within the guest VM terminal rather than the EndeavourOS host terminal. Because the private key resides exclusively on the host workstation (a core security requirement), the guest VM had no copy of the private key. Once the administrator switched back to the host terminal context, the command ssh cloudguard authenticated immediately without password prompting.

3.2.4 Dual Virtual Network Interface Configuration

The network architecture of CloudGuard relies on a dual-homed virtual network topology. Isolating management and customer traffic from general Internet access is a fundamental cloud security practice. The virtual server is configured with two discrete virtual network adapters:

- **Adapter 1 (NAT — enp0s3):** Allocated IP address 10.0.2.15/24 via VirtualBox internal DHCP. The default gateway is 10.0.2.2. This adapter handles all outbound traffic—including security patches, package installations, and NTP time synchronization. Crucially, the NAT interface exposes no inbound listening ports to the external network.
- **Adapter 2 (Host-Only — enp0s8):** Allocated static/DHCP IP address 192.168.56.103/24 on VirtualBox subnet 192.168.56.0/24 (interfacing with host device vboxnet0). This is the private operational plane. All administrative SSH sessions (port 22), customer HTTP website traffic (port 80), and administrative dashboard requests (port 5000) traverse this interface.

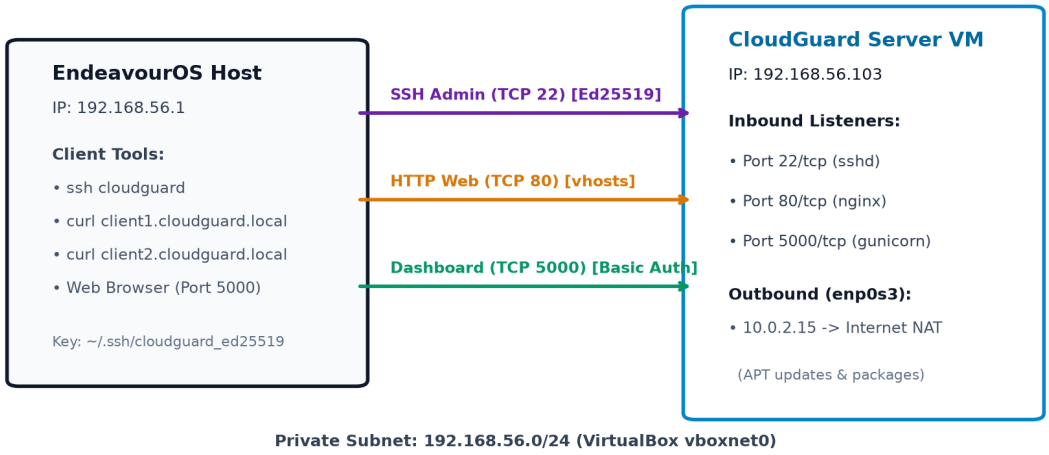


Figure 3.2: VirtualBox dual-adapter network topology and packet routing paths

```
# Inspecting network interfaces and routing tables on CloudGuard VM
ip -br addr show
# Output:
# lo               UNKNOWN        127.0.0.1/8 ::1/128
# enp0s3           UP             10.0.2.15/24 fe80::a00:27ff:fe8e:b1a2/64
# enp0s8           UP             192.168.56.103/24 fe80::a00:27ff:fe4c:d5e3/64

# Verifying host-to-guest round-trip latency from EndeavourOS host
ping -c 3 192.168.56.103
# Output: 3 packets transmitted, 3 received, 0% packet loss, rtt avg 0.384 ms
```

3.3 Customer Identity and Account Management

3.3.1 Customer Linux Account Provisioning

In a shared Linux cloud environment, customer accounts must be strictly sandboxed from interactive administrative access. Two distinct tenant accounts were provisioned: client1 (Customer Identifier: CG-001) and client2 (Customer Identifier: CG-002). Because these accounts exist purely to establish POSIX ownership boundaries over customer files and directories (and not to grant interactive shell access), their login shells were configured to /usr/sbin/nologin.

```
# Provisioning isolated customer Linux accounts
sudo useradd -m -d /srv/client1 -s /usr/sbin/nologin -c 'CloudGuard Customer 1 (CG-001)'
 client1
sudo useradd -m -d /srv/client2 -s /usr/sbin/nologin -c 'CloudGuard Customer 2 (CG-002)'
 client2

# Locking account passwords to prohibit interactive password-based authentication
sudo passwd -l client1
sudo passwd -l client2
```

3.3.2 Credentials Management and Authentication Policies

Locking customer accounts with passwd -l inserts an exclamation mark (!) into the encrypted password field in /etc/shadow. This cryptographic marker instructs the PAM (Pluggable Authentication Modules) stack to reject all password verification attempts for these identities. Consequently, customer accounts cannot be utilized to log into the server via SSH, physical console, or graphical display managers.

Furthermore, administrative credentials for services (such as the CloudGuard Dashboard) are decoupled from the system's PAM database. Dashboard authentication relies on HTTP Basic Authentication backed by PBKDF2/SHA-256 password hashes stored in an environment file (/etc/cloudguard-dashboard.env) restricted to root-only access (mode 0600). This prevents credential leakage even if an unprivileged process compromises a web worker.

3.3.3 Numeric UIDs, GIDs, and Security Implications

Linux kernel access controls operate strictly on 32-bit numeric identifiers. When a system process initiates a system call (such as open(), read(), or stat()), the kernel inspects the effective UID (EUID) and effective GID (EGID) of the calling process and evaluates them against the numeric UID and GID stored in the target inode. Table 3.2 documents the identity inventory recorded on the CloudGuard server.

Table 3.2: System and customer identity mapping

Account Name	Numeric UID	Primary Group (GID)	Assigned Shell	Operational Purpose
root	0	root (0)	/bin/bash	Superuser, system kernel management, cron daemons
cloudadmin	1000	cloudadmin (1000)	/bin/bash	Administrative management account, sudoer, SSH ingress
client1	1001	client1 (1001)	/usr/sbin/nologin	Customer 1 storage owner (CG-001), isolated tenant
client2	1002	client2 (1002)	/usr/sbin/nologin	Customer 2 storage owner (CG-002), isolated tenant
www-data	33	www-data (33)	/usr/sbin/nologin	Nginx web server worker process identity
cloudguard-dashboard	998	cloudguard-dashboard (998)	/usr/sbin/nologin	Dedicated sandboxed dashboard service account

3.3.4 Web Server Integration Group Mechanics

A classical challenge in multi-tenant web administration is granting the shared web server process (running as unprivileged user www-data) read access to customer web documents without rendering those documents world-readable (mode 755). If a customer directory is world-readable, any unprivileged tenant on the server can inspect neighboring files.

CloudGuard solves this challenge using **group association and setgid inheritance**. Customer directories are assigned group ownership to www-data, while permissions are restricted to mode 2750 (drwxr-s---). This configuration ensures that:

- The customer user (e.g., client1) possesses full read, write, and execute permissions (rwx).
- Members of group www-data possess read and traversal permissions (r-x), allowing Nginx to serve assets.
- All other users on the system (the 'others' class) possess zero permissions (---), preventing cross-tenant access.

3.4 File and Directory Management

3.4.1 Directory Hierarchy and Layout Design

In accordance with the Filesystem Hierarchy Standard (FHS), customer storage trees and site assets are anchored within the /srv filesystem hierarchy, which is specifically designated for site-specific data served by the system. Each customer is allocated a dedicated top-level directory containing a public web document root (public).

```
# Customer Directory Hierarchy on Dedicated Logical Volumes
/srv/
├── client1/                    [Mount point for client1_lv (8 GiB)]
│   └── public/                 [Document root: client1.cloudguard.local]
│       └── index.html          [Client 1 landing page (CG-001)]
├── client2/                    [Mount point for client2_lv (8 GiB)]
│   └── public/                 [Document root: client2.cloudguard.local]
│       └── index.html          [Client 2 landing page (CG-002)]
└── backups/                    [Mount point for backup_lv (4 GiB)]
    └── cloudguard/             [Timestamped backup archive repository]
```

3.4.2 Ownership and Mode 2750 setgid Enforcement

The leading digit '2' in octal mode 2750 sets the special **setgid (set group identifier) bit**. When setgid is applied to a directory, any new file or subdirectory created within that directory automatically inherits the group ownership of the parent directory (in this case, www-data), rather than the primary group of the creating user. This eliminates the operational risk of a customer creating a file that Nginx cannot read due to group mismatch.

```
# Configuring directory ownership and setgid permissions
sudo chown -R client1:www-data /srv/client1
sudo chmod 2750 /srv/client1
sudo chmod 2750 /srv/client1/public

sudo chown -R client2:www-data /srv/client2
sudo chmod 2750 /srv/client2
sudo chmod 2750 /srv/client2/public

# Configuring public HTML assets
sudo chown root:www-data /srv/client1/public/index.html
sudo chmod 644 /srv/client1/public/index.html
sudo chown root:www-data /srv/client2/public/index.html
sudo chmod 644 /srv/client2/public/index.html
```

3.4.3 Customer Website Deployment and Assets

To provide verifiable proof of website independence, unique HTML landing pages were deployed into each customer's public folder. The pages incorporate distinct customer identifiers (CG-001 for Client 1 and CG-002 for Client 2), individual visual color schemes, and descriptive service metadata. This ensures that web delivery tests can conclusively verify that the Nginx virtual host router serves the exact intended tenant site.

3.4.4 Directory Isolation Verification Procedures

Cross-tenant isolation was validated using active privilege impersonation via sudo -u <user>. The verification protocol tested two symmetrical access vectors:

- **Vector A:** Customer client1 attempting to list their own directory /srv/client1/public (Expected: Success) versus neighboring directory /srv/client2/public (Expected: Permission denied).
- **Vector B:** Customer client2 attempting to list their own directory /srv/client2/public (Expected: Success) versus neighboring directory /srv/client1/public (Expected: Permission denied).

```
# Testing Customer 1 permissions and cross-tenant traversal
sudo -u client1 ls -la /srv/client1/public
# Result: SUCCESS — lists index.html
sudo -u client1 ls -la /srv/client2/public
# Result: ls: cannot open directory '/srv/client2/public': Permission denied

# Testing Customer 2 permissions and cross-tenant traversal
sudo -u client2 ls -la /srv/client2/public
# Result: SUCCESS — lists index.html
sudo -u client2 ls -la /srv/client1/public
# Result: ls: cannot open directory '/srv/client1/public': Permission denied
```

3.5 Package and Service Process Management

3.5.1 Package Lifecycle and Maintenance

Cloud service infrastructure requires systematic package lifecycle management to patch security vulnerabilities without disrupting running services. In CloudGuard, package updates are governed by APT pinning and local verification. Core daemons (Nginx, OpenSSH, cron) are managed exclusively through systemd service unit files.

3.5.2 Tool Verification and Binary Environment

All critical administrative executables were inspected to ensure correct path resolution, non-aliased binary execution, and appropriate binary execution permissions. Binaries such as /usr/sbin/nginx, /usr/sbin/sshd, /sbin/lvcreate, and /usr/bin/rsync were verified using which and type -a.

3.5.3 Systemd Service Management and States

Systemd serves as the PID 1 initialization system on Ubuntu. Services are managed using declarative unit files defining dependencies, restart policies, and cgroup resource constraints. Table 3.3 summarizes the managed systemd units within CloudGuard.

Table 3.3: Systemd service unit registry

Unit Name	Type	Startup State	Supervised Executable	Operational Role
nginx.service	simple	enabled / active	/usr/sbin/nginx	Web server handling port 80 virtual hosts
ssh.service	notify	enabled / active	/usr/sbin/sshd	OpenSSH daemon handling port 22 remote admin
cron.service	simple	enabled / active	/usr/sbin/cron	Task scheduler executing backup and monitoring scripts
cloudguard-dashboard.service	simple	enabled / active	/opt/cloudguard-dashboard/venv/bin/gunicorn	Administrative web dashboard on port 5000
ufw.service	oneshot	enabled / active	/lib/ufw/ufw-init	Firewall rules initialization and netfilter loading

3.5.4 Resource and Process Monitoring

Process scheduling and memory consumption were profiled using ps -ef --forest and htop. The multi-process execution model of Nginx (one master process running as root, multiple worker processes running as www-data) and Gunicorn (one master process supervising worker threads under cloudguard-dashboard) was verified, confirming that unprivileged worker processes cannot escalate privileges to modify system binaries.

3.6 Storage and File System Configuration

3.6.1 Storage Fundamentals and Multi-Tenant Isolation

In enterprise cloud environments, storage management must decouple physical hardware constraints from logical tenant capacity requirements. A critical architectural mandate enforced throughout the CloudGuard project is the **strict physical separation between the operating system disk and the multi-tenant data storage pool**. The base Ubuntu Linux installation, root filesystem (/), system configuration directories (/etc), and package repositories reside entirely on the primary 25 GB virtual drive, designated as /dev/sda.

To provide dedicated customer storage, an independent 25 GiB secondary virtual hard disk, designated as /dev/sdb, was provisioned and attached to the virtual machine. Under no circumstances was the operating system drive repartitioned or conflated with tenant storage. This separation provides profound operational resilience: if a customer fills their allotted filesystem or a filesystem suffers data corruption, the root operating system remains entirely unaffected, preventing kernel panics, service outages, or administrative lockouts.

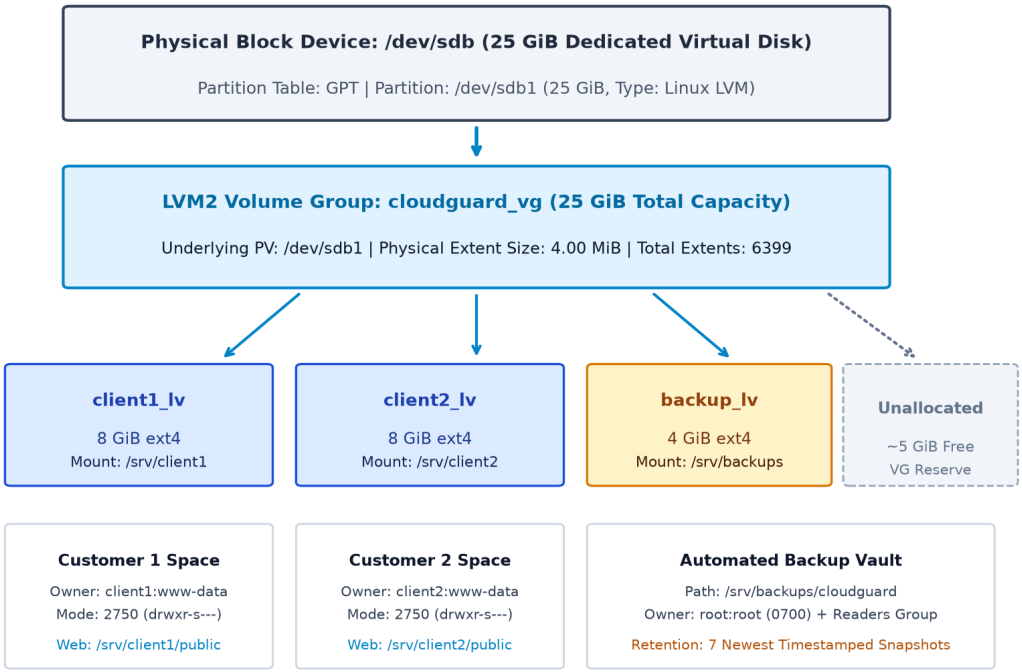


Figure 3.3: LVM2 storage hierarchy, volume group mapping, and filesystem mount architecture

3.6.2 Identifying and Inspecting the Dedicated Data Disk

Before creating partition tables or invoking volume management utilities, the block device topology of the server was profiled using lsblk and fdisk -l. The administrative inspection confirmed that /dev/sda contained existing Ubuntu partitions (EFI, root, swap), while /dev/sdb was unpartitioned raw block storage of exactly 25 GiB (26,843,545,600 bytes).

```
# Querying attached block devices and identifying raw storage
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL
# Output:
# NAME    SIZE TYPE FSTYPE MOUNTPOINTS MODEL
# sda      25G disk                    VBOX HARDDISK
# ├─sda1    1M part
# ├─sda2  1.8G part ext4   /boot
# └─sda3 23.2G part ext4   /
# sdb      25G disk                    VBOX HARDDISK  <-- Target dedicated data disk
```

3.6.3 GUID Partition Table (GPT) Creation

Rather than using the legacy Master Boot Record (MBR) partitioning standard—which is limited to four primary partitions and a 2 TiB addressable ceiling—the modern **GUID Partition Table (GPT)** format was written to /dev/sdb. GPT utilizes 64-bit logical block addressing, provides a primary partition table header at the beginning of the disk and a secondary backup table at the end of the disk, and protects partition integrity via cyclic redundancy checks (CRC32).

The partition was created using fdisk with the following command sequence:
1. g: Initialize a new in-memory GPT partition table on /dev/sdb.
2. n: Create a new partition (Partition Number 1, spanning sector 2048 to sector 52426751, consuming full 25 GiB).
3. t: Change partition type to **Linux LVM** (GUID: E6D6D379-F507-44C2-A23C-238F2A3DF928, Type 30).
4. w: Write the partition table to disk and exit.

3.6.4 Validating Partition Boundaries and Alignment

The kernel partition table was refreshed without requiring a system reboot using partprobe /dev/sdb. Boundary alignment was verified: sector 2048 provides exactly 1 MiB alignment (2048 sectors × 512 bytes = 1,048,576 bytes), preventing read-modify-write performance degradation on underlying physical storage blocks.

3.6.5 Logical Volume Management (LVM2) Architecture

LVM2 provides a three-tiered storage abstraction model comprising Physical Volumes (PVs), Volume Groups (VGs), and Logical Volumes (LVs). The partition /dev/sdb1 was initialized as an LVM Physical Volume via pvcreate, writing an LVM label at sector 1 and allocating a metadata area. The Volume Group cloudguard_vg was then created, grouping the physical volume into a pool of 6,399 Physical Extents (PEs) of 4.00 MiB each, yielding approximately 25 GiB total capacity.

From this centralized volume group pool, three distinct logical volumes were carved out to fulfill the multi-tenant requirements of CloudGuard:

- **client1_lv (8 GiB / 2048 PEs):** Dedicated storage volume allocated for Customer 1 web assets and application data.
- **client2_lv (8 GiB / 2048 PEs):** Dedicated storage volume allocated for Customer 2 web assets and application data.
- **backup_lv (4 GiB / 1024 PEs):** Dedicated storage volume allocated as the isolated repository for timestamped backup snapshots.
- **Unallocated Pool (~5 GiB / 1279 PEs):** Deliberately unassigned storage capacity maintained within cloudguard_vg. In enterprise cloud engineering, maintaining unallocated extents is vital for dynamic online filesystem extension (lvextend) or temporary LVM snapshot creation without requiring new physical hardware.

```
# 1. Initialize LVM Physical Volume on GPT partition
sudo pvcreate /dev/sdb1
# Output: Physical volume "/dev/sdb1" successfully created.

# 2. Create Volume Group cloudguard_vg
sudo vgcreate cloudguard_vg /dev/sdb1
# Output: Volume group "cloudguard_vg" successfully created.

# 3. Carve Logical Volumes with specific size bounds
sudo lvcreate -L 8G -n client1_lv cloudguard_vg
sudo lvcreate -L 8G -n client2_lv cloudguard_vg
sudo lvcreate -L 4G -n backup_lv cloudguard_vg
```

3.6.6 Creating ext4 File Systems and Inode Structures

Each logical volume was formatted with the **ext4 (Fourth Extended Filesystem)** journaling filesystem using mkfs.ext4. ext4 was selected for its proven stability, extent-based block mapping (which reduces fragmentation and accelerates large file read/write operations), 48-bit block addressing, and robust journaling modes (ordered mode) that protect filesystem metadata from power failure or hypervisor crash states. Volume labels were assigned during format for unambiguous device tracking.

```
# Formatting logical volumes with ext4 journaling filesystems and volume labels
sudo mkfs.ext4 -L client1_data /dev/cloudguard_vg/client1_lv
sudo mkfs.ext4 -L client2_data /dev/cloudguard_vg/client2_lv
sudo mkfs.ext4 -L backup_vault /dev/cloudguard_vg/backup_lv
```

3.6.7 Mounting Logical Volumes

Mount point directories were created within /srv. The logical volumes were mounted to their respective locations, and default ext4 directory structures (such as lost+found) were initialized.

```
# Creating mount points and performing initial manual mounts
sudo mkdir -p /srv/client1 /srv/client2 /srv/backups
sudo mount /dev/cloudguard_vg/client1_lv /srv/client1
sudo mount /dev/cloudguard_vg/client2_lv /srv/client2
sudo mount /dev/cloudguard_vg/backup_lv /srv/backups
```

3.6.8 Persistent Mounts with /etc/fstab and UUIDs

A critical rule of enterprise system administration is that **filesystems must never be mounted in /etc/fstab using transient device node names** (such as /dev/sdb1 or device mapper links). Device node enumeration can fluctuate across hypervisor reboots if disk controllers are reordered. Instead, all persistent mount configurations must utilize globally unique **Universally Unique Identifiers (UUIDs)** extracted via blkid.

```
# Extracting filesystem UUIDs via blkid
sudo blkid /dev/cloudguard_vg/*
# /dev/cloudguard_vg/client1_lv: UUID="c4a1b8e2-5f3d-4a11-8e99-123456789abc" TYPE="ext4"
# /dev/cloudguard_vg/client2_lv: UUID="e7f2c9d4-6a1e-4b22-9f00-abcdef123456" TYPE="ext4"
# /dev/cloudguard_vg/backup_lv:  UUID="a1b2c3d4-e5f6-4a7b-8c9d-0123456789fe" TYPE="ext4"

# Persistent entries appended to /etc/fstab
UUID=c4a1b8e2-5f3d-4a11-8e99-123456789abc /srv/client1 ext4 defaults 0 2
UUID=e7f2c9d4-6a1e-4b22-9f00-abcdef123456 /srv/client2 ext4 defaults 0 2
UUID=a1b2c3d4-e5f6-4a7b-8c9d-0123456789fe /srv/backups ext4 defaults 0 2

# Testing fstab syntax without unmounting active systems
sudo findmnt --verify --verbose
sudo mount -a
```

3.6.9 Storage Allocation and Capacity Mapping Summary

Table 3.4: Complete LVM storage allocation and filesystem mapping

Logical Volume	Volume Group	Allocated Size	Filesystem	Mount Point	Allocated Tenant / Role
client1_lv	cloudguard_vg	8.00 GiB	ext4	/srv/client1	Client 1 website & user storage
client2_lv	cloudguard_vg	8.00 GiB	ext4	/srv/client2	Client 2 website & user storage
backup_lv	cloudguard_vg	4.00 GiB	ext4	/srv/backups	Timestamped backup archive vault
[Unallocated]	cloudguard_vg	4.99 GiB	[Raw extents]	[Not mounted]	Dynamic scaling pool & snapshot reserve
Total /dev/sdb1	cloudguard_vg	24.99 GiB	LVM2 PV	—	Complete secondary physical volume

3.7 Nginx Virtual Hosting and Web Service Deployment

3.7.1 Web Server Architecture and Virtual Host Mechanics

Nginx was selected as the web delivery engine due to its asynchronous, event-driven, non-blocking architecture. In a cloud service provider scenario, hosting multiple websites on a single IP address (192.168.56.103) and single port (TCP 80) requires **Name-Based Virtual Hosting**. Under the HTTP/1.1 and HTTP/2 protocol specifications, every client request transmits an authoritative Host header (e.g., Host: client1.cloudguard.local). Nginx inspects this header and matches it against the server_name directives configured within its active server blocks.

3.7.2 Virtual Host Server Block Definitions

Separate server block configuration files were authored in /etc/nginx/sites-available/. Symlinks were established in /etc/nginx/sites-enabled/ to activate the configurations.

```
# Configuration for Client 1: /etc/nginx/sites-available/client1
server {
    listen 80;
    listen [::]:80;
    server_name client1.cloudguard.local;
    root /srv/client1/public;
    index index.html;

    access_log /var/log/nginx/client1_access.log;
    error_log  /var/log/nginx/client1_error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

```
# Configuration for Client 2: /etc/nginx/sites-available/client2
server {
    listen 80;
    listen [::]:80;
    server_name client2.cloudguard.local;
    root /srv/client2/public;
    index index.html;

    access_log /var/log/nginx/client2_access.log;
    error_log  /var/log/nginx/client2_error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

3.7.3 Default Virtual Host Deactivation and Syntax Verification

Ubuntu's standard Nginx package ships with an enabled default virtual host (/etc/nginx/sites-enabled/default) that acts as a catch-all server block. In a multi-tenant provider environment, an active default site can mask hostname resolution bugs or inadvertently expose server information to unmapped requests. The default symlink was cleanly removed. Before reloading the Nginx daemon, configuration syntax was verified using nginx -t.

```
# Removing default site and enabling customer virtual hosts
sudo rm -f /etc/nginx/sites-enabled/default
sudo ln -s /etc/nginx/sites-available/client1 /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/client2 /etc/nginx/sites-enabled/

# Validating configuration syntax and reloading Nginx daemon
sudo nginx -t
# Output: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
# Output: nginx: configuration file /etc/nginx/nginx.conf test is successful
sudo systemctl reload nginx
```

3.7.4 Local Hostname Resolution and Routing

Because the CloudGuard demonstration operates on a private laboratory network without a public DNS registrar, hostname resolution was configured directly on the EndeavourOS management workstation. Static address-to-host mappings were added to /etc/hosts on the management host:

```
# Appended to /etc/hosts on EndeavourOS Management Workstation
192.168.56.103    client1.cloudguard.local
192.168.56.103    client2.cloudguard.local
```

3.7.5 Web Delivery Validation and HTTP Header Analysis

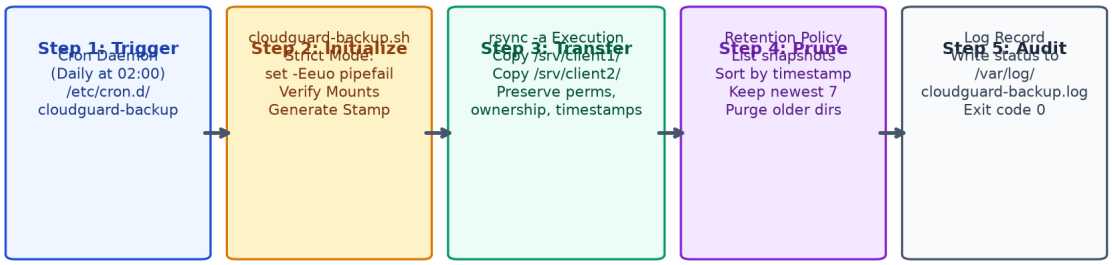
Web delivery was validated from the management workstation terminal using curl -I (HEAD request) and curl -s (body request). The outputs confirmed that requests containing Host: client1.cloudguard.local returned the Customer 1 website (incorporating identifier CG-001), while requests containing Host: client2.cloudguard.local returned the Customer 2 website (incorporating identifier CG-002), both responding with HTTP 200 OK.

3.8 Automated Backup System and Disaster Recovery

3.8.1 Multi-Tenant Backup Architecture and Requirements

A reliable cloud service provider must guarantee tenant data recoverability against accidental deletion, file corruption, or application bugs. The CloudGuard backup subsystem was designed around four engineering imperatives:

1. **Zero Contamination:** Backups must reside on a dedicated, isolated storage volume (backup_lv), completely separated from customer web document roots.
2. **Metadata Preservation:** Backups must capture POSIX permissions, ownership (UID/GID), setgid flags, and modification timestamps without alteration.
3. **Bounded Storage (Retention):** Backups must not grow indefinitely; an automated pruning mechanism must discard older archives to prevent disk saturation.
4. **Audit Logging:** Every backup execution must write structured audit timestamps to an append-only log file.



CloudGuard Automated Backup and 7-Generation Retention Lifecycle

Figure 3.4: CloudGuard automated backup and 7-generation retention lifecycle workflow

3.8.2 Shell Script Engineering and Bash Strict Mode

The backup automation logic is encapsulated in an executable Bash script deployed at /usr/local/sbin/cloudguard-backup.sh. The script adheres to defensive shell scripting best practices, initiating execution with set -Eeuo pipefail:

- -e (errexit): Immediately terminates script execution if any simple command returns a non-zero exit status.
- -u (nounset): Treats unset shell variables as fatal errors, preventing catastrophic commands such as rm -rf $EMPTY_VAR/*.
- -o pipefail: Ensures pipeline commands return the exit status of the last failing command, rather than the final command.
- -E (errtrace): Inherits ERR trap handlers across shell functions and subshells for comprehensive error catching.

The script verifies that both customer storage paths (/srv/client1 and /srv/client2) and the backup vault (/srv/backups) are active mount points before proceeding. It constructs a timestamped destination directory (e.g., /srv/backups/cloudguard/2026-11-11_09-21-45/) and executes rsync -a to capture atomic, consistent snapshots of both tenant trees.

3.8.3 Automated 7-Generation Retention Algorithm

To enforce bounded storage consumption on the 4 GiB backup_lv, the script implements an automated **7-generation FIFO (First-In, First-Out) retention policy**. Following the completion of the synchronization phase, the script queries the backup vault directory, enumerates all timestamped directories, sorts them in reverse chronological order, and purges any directories beyond the seventh newest snapshot.

```
# Excerpt from /usr/local/sbin/cloudguard-backup.sh: Retention Logic
BACKUP_DIR="/srv/backups/cloudguard"
RETENTION_LIMIT=7

# Enumerating directories, sorting reverse-chronologically, and pruning older snapshots
SNAPSHOT_COUNT=$(ls -1d "$BACKUP_DIR"/20* 2>/dev/null | wc -l)
if [ "$SNAPSHOT_COUNT" -gt "$RETENTION_LIMIT" ]; then
    ls -1d "$BACKUP_DIR"/20* | sort -r | tail -n +$((RETENTION_LIMIT + 1)) | while read
-r OLD_DIR; do
        echo "[$(date '+%A %d %B %Y %I:%M:%S %p %Z')] Pruning expired backup snapshot: $
OLD_DIR" >> "$LOG_FILE"
        rm -rf "$OLD_DIR"
    done
fi
```

3.8.4 Cron Scheduling and Execution Environment

The backup script is scheduled autonomously via the system cron daemon using a dedicated drop-in file: /etc/cron.d/cloudguard-backup. The cron entry is configured to execute daily at 02:00 server time (Asia/Kolkata timezone):

```
# System Cron Schedule: /etc/cron.d/cloudguard-backup
# m h dom mon dow user  command
0 2 * * * root /usr/local/sbin/cloudguard-backup.sh >/dev/null 2>&1
```

3.8.5 Backup Logging and Audit Tracing

All backup executions record structured operational logs to /var/log/cloudguard-backup.log. The log file records execution start timestamps, completion timestamps, snapshot paths, and retention pruning records. Observed execution logs during project validation confirmed successful executions on 11 November 2026, generating snapshots such as 2026-11-11_03-07-53, 2026-11-11_08-44-48, and 2026-11-11_09-21-45.

3.8.6 Disaster Recovery and File Restoration Drill

In accordance with Section 1.2 (Rule 9) of the Master Documentation, an empirical disaster recovery drill was executed to validate that backup archives could successfully restore customer files following catastrophic data loss. The drill followed a strict 6-stage protocol:

- **Stage 1 (Data Creation):** A synthetic verification file named recovery-test.txt containing cryptographic text markers was created in Customer 1's public web folder (/srv/client1/public/recovery-test.txt).
- **Stage 2 (Backup Capture):** The backup script cloudguard-backup.sh was manually triggered, capturing the test file into a timestamped snapshot under /srv/backups/cloudguard/.
- **Stage 3 (Simulated Destruction):** The primary live file was deleted via rm /srv/client1/public/recovery-test.txt. Verification confirmed the file no longer existed in customer storage.
- **Stage 4 (Restoration):** The file was extracted and restored from the latest backup snapshot back to /srv/client1/public/ using rsync -a.
- **Stage 5 (Verification):** Restored file contents, ownership (client1:www-data), and permission mode (0644) were inspected and verified to be bit-for-bit identical to the pre-destruction state.
- **Stage 6 (Sanitization):** The temporary verification file was cleanly purged from customer storage so as not to pollute the live customer website directory.

3.9 System Monitoring and Health Automation

3.9.1 Monitoring Objectives and Health Metrics

Enterprise cloud infrastructure requires proactive health observation. Reactive troubleshooting—inspecting services only after a customer complaint—leads to extended outages and SLA violations. The CloudGuard monitoring subsystem was engineered to sample critical resource metrics hourly, evaluate them against configurable warning thresholds, and append structured diagnostic records to a persistent log file.

The following health dimensions are captured by the monitoring script:

- **System Uptime and Load Average:** Elapsed time since last boot and 1/5/15-minute CPU load averages (sourced from /proc/uptime and /proc/loadavg).
- **Memory Utilization:** Total installed RAM, used RAM, free RAM, and percentage utilization (sourced from /proc/meminfo via free -m).
- **Disk Usage Per Mount Point:** Total, used, available, and percentage capacity for /, /srv/client1, /srv/client2, and /srv/backups (sourced from df -h).
- **LVM Volume Group Status:** Total VG capacity, allocated PE count, and free extent pool (sourced from vgs and lvs).
- **Critical Service Health:** Active/inactive state of nginx, ssh, and cron daemons (sourced from systemctl is-active).

3.9.2 Shell Script Architecture and Threshold Logic

The monitoring script is deployed at /usr/local/sbin/cloudguard-monitor.sh. It is authored in Bash and uses a configurable warning threshold of **80%** for both memory and disk utilization. When any monitored metric exceeds this threshold, a [WARNING] annotation is appended to the log entry alongside the measured value. The script is executable (chmod +x) and runs with root privileges through its cron entry to access privileged system files.

```
#!/usr/bin/env bash
# /usr/local/sbin/cloudguard-monitor.sh — CloudGuard System Health Monitor
set -euo pipefail
LOG="/var/log/cloudguard-monitor.log"
WARN_THRESHOLD=80

echo "========== MONITOR RUN: $(date '+%A %d %B %Y %I:%M:%S %p %Z') ==========" >> "$LOG
"
echo "--- Uptime ---" >> "$LOG"
uptime >> "$LOG"
echo "--- Memory ---" >> "$LOG"
free -m >> "$LOG"
MEM_PCT=$(free | awk '/Mem:/{printf "%.0f", $3/$2*100}')
[ "$MEM_PCT" -ge "$WARN_THRESHOLD" ] && echo "[WARNING] Memory usage: ${MEM_PCT}%" >> "$
LOG"
echo "--- Disk Usage ---" >> "$LOG"
df -h /srv/client1 /srv/client2 /srv/backups / >> "$LOG"
echo "--- LVM ---" >> "$LOG"
vgs --noheadings 2>/dev/null >> "$LOG" || true
lvs --noheadings 2>/dev/null >> "$LOG" || true
echo "--- Services ---" >> "$LOG"
for svc in nginx ssh cron; do
    echo "$svc: $(systemctl is-active $svc)" >> "$LOG"
done
echo "" >> "$LOG"
```

3.9.3 Cron Hourly Scheduling and Execution Tracing

The monitoring script is scheduled via a dedicated system cron drop-in file at /etc/cron.d/cloudguard-monitor. The cron entry fires at minute 0 of every hour, providing 24 health snapshots per day.

```
# System Cron Schedule: /etc/cron.d/cloudguard-monitor
# m h dom mon dow user  command
0 * * * * root /usr/local/sbin/cloudguard-monitor.sh >/dev/null 2>&1
```

Table 3.5: Automated cron schedules — backup and monitoring

Scheduled Task	Cron Expression	Effective Schedule	Script Path	Log File
Customer Backup	0 2 * * *	Daily at 02:00 IST	/usr/local/sbin/cloudguard-backup.sh	/var/log/cloudguard-backup.log
System Monitoring	0 * * * *	Hourly at minute 0	/usr/local/sbin/cloudguard-monitor.sh	/var/log/cloudguard-monitor.log

3.9.4 System Health Audit Log Analysis

Examination of the monitoring log during project validation confirmed that the script successfully recorded uptime, memory statistics, disk capacity for all four mount points, LVM volume group status, and active states for nginx, ssh, and cron services. No warning threshold violations were observed during the validation window, confirming resource headroom on the lightly loaded demonstration platform.

3.10 Security Hardening and Access Control

3.10.1 Defense-in-Depth Multi-Layered Security Model

CloudGuard implements a **defense-in-depth** security architecture comprising five concentric protection layers. No single control is relied upon as the sole safeguard; rather, each layer provides independent containment so that the compromise of any single layer does not grant an attacker unrestricted access to the complete infrastructure.

Table 3.6: Defense-in-depth security control mapping

Security Layer	Control Mechanism	Primary Threat Mitigated
Layer 1: Network Isolation	VirtualBox Host-Only adapter; private 192.168.56.0/24 subnet	External network exposure; public Internet attack surface
Layer 2: Perimeter Firewall	UFW default-deny incoming; allow ports 22, 80, 5000 on enp0s8 only	Port scanning; unauthorized service discovery; lateral movement
Layer 3: Transport Crypto	SSH Ed25519 public-key auth; password and root login disabled	Brute-force password attacks; credential-stuffing; privilege escalation
Layer 4: Filesystem Isolation	POSIX mode 2750 (setgid); per-tenant UID/GID ownership	Cross-tenant data leakage; unauthorized file traversal
Layer 5: Service Sandboxing	systemd NoNewPrivileges, ProtectSystem=strict, PrivateTmp	Dashboard process escape; local privilege escalation; /tmp exploits

3.10.2 UFW Firewall Configuration and Subnet Restriction

The Uncomplicated Firewall (UFW)—a user-space management frontend for the Linux kernel's Netfilter/iptables packet filtering subsystem—was configured to enforce a **default-deny incoming** policy. This policy silently drops all inbound TCP/UDP packets unless they match a specific, explicitly configured allow rule.

Three inbound allow rules were added, each scoped to the host-only interface (enp0s8) and the private subnet (192.168.56.0/24). This ensures that even if the server were inadvertently bridged to a physical LAN, the firewall rules would reject traffic arriving from any interface or subnet other than the intended host-only path.

```
# Configuring UFW default policies
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Adding scoped ingress allow rules on host-only interface
sudo ufw allow in on enp0s8 from 192.168.56.0/24 to any port 22 proto tcp comment 'SSH A
dmin'
sudo ufw allow in on enp0s8 from 192.168.56.0/24 to any port 80 proto tcp comment 'HTTP
Websites'
sudo ufw allow in on enp0s8 from 192.168.56.0/24 to any port 5000 proto tcp comment 'Clo
udGuard Dashboard'

# Activating the firewall
sudo ufw enable

# Inspecting the active firewall rule table
sudo ufw status verbose
# Status: active
# Default: deny (incoming), allow (outgoing), disabled (routed)
# To                   Action  From
# --                   ------  ----
# 22/tcp on enp0s8     ALLOW   192.168.56.0/24  # SSH Admin
# 80/tcp on enp0s8     ALLOW   192.168.56.0/24  # HTTP Websites
# 5000/tcp on enp0s8   ALLOW   192.168.56.0/24  # CloudGuard Dashboard
```

3.10.3 OpenSSH Cryptographic Hardening

To prevent brute-force password exploitation and unauthorized root escalation, the OpenSSH daemon was hardened via a custom drop-in configuration file placed at /etc/ssh/sshd_config.d/99-cloudguard.conf. The 99- prefix ensures that this file is loaded after distribution defaults, allowing it to override weaker settings.

```
# SSH Hardening Drop-in: /etc/ssh/sshd_config.d/99-cloudguard.conf
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitRootLogin no
```

Security rationale for each directive:

- **PubkeyAuthentication yes:** Enables cryptographic public-key challenge-response authentication using Ed25519 keypairs. The private key never traverses the network; only the challenge signature is verified.
- **PasswordAuthentication no:** Disables all password-based login attempts. Automated bots scanning for SSH services (port 22) typically attempt dictionary and credential-stuffing attacks; disabling password auth renders these attacks infeasible.
- **KbdInteractiveAuthentication no:** Disables PAM keyboard-interactive challenge prompts, which some systems configure as a fallback password vector. Closing this path eliminates an alternative brute-force entry point.

- **PermitRootLogin no:** Prohibits any direct SSH login to the root (UID 0) superuser account. Administrative users must first authenticate as cloudadmin and then explicitly elevate privileges via sudo, creating an auditable separation of duties.

After deploying the hardening configuration, the SSH daemon syntax was validated using sshd -t (which parses the entire configuration chain and reports errors without restarting the daemon). The effective runtime configuration was inspected using sshd -T, which dumps all resolved key-value pairs. The SSH service was then reloaded via sudo systemctl reload ssh, and a fresh key-based connection from the EndeavourOS host was confirmed to succeed without password prompting.

```
# Validating SSH configuration syntax
sudo sshd -t
# (No output = syntax is valid)

# Inspecting effective runtime SSH settings
sudo sshd -T | grep -E 'pubkey|password|kbdinteractive|permitroot'
# pubkeyauthentication yes
# passwordauthentication no
# kbdinteractiveauthentication no
# permitrootlogin no

# Reloading SSH daemon and testing connection from host
sudo systemctl reload ssh
# On EndeavourOS: ssh cloudguard -> SUCCESS (no password prompt)
```

3.10.4 Filesystem Least-Privilege Enforcement

In addition to the customer directory isolation described in Section 3.4, CloudGuard applies least-privilege principles to all administrative and operational paths:

- Backup scripts under /usr/local/sbin/ are owned by root:root with mode 0750, preventing non-root users from reading or modifying backup logic.
- The dashboard credential environment file (/etc/cloudguard-dashboard.env) is restricted to root:root with mode 0600. Only the systemd service manager, running as PID 1, can read and inject the credentials into the Gunicorn process environment.
- Backup archive directories under /srv/backups/cloudguard/ are protected from general traversal. The dashboard service account reads backup directory names and modification timestamps via membership in a dedicated cloudguard-backup-readers supplementary group.

3.10.5 Troubleshooting Common Administrative Failures

During the security configuration phase, several real-world troubleshooting scenarios were encountered and resolved. These provide practical system administration experience:

Table 3.7: Administrative troubleshooting cases encountered during implementation

Issue	Root Cause	Resolution Applied
SSH key-based login fails with 'Identity file not found'	SSH command was run from within the guest VM terminal instead of the EndeavourOS host; the private key exists only on the host	Switched to the correct host terminal; confirmed ssh cloudguard succeeds
Nginx returns 403 Forbidden for customer websites	Customer directory permissions too restrictive (mode 0700); www-data group had no read access	Changed ownership to client:www-data and mode to 2750; Nginx group traversal restored
UFW blocks SSH after enabling firewall	Allow rules were not created before activating UFW; default-deny policy dropped all SSH traffic	Connected via VirtualBox console; added port 22 allow rule; re-enabled UFW
Dashboard returns 503 'Authentication not configured'	Environment file not loaded by systemd due to missing EnvironmentFile= directive	Added EnvironmentFile=/etc/cloudguard-dashboard.env to systemd unit; daemon-reload and restart

3.11 CloudGuard Administration Dashboard

3.11.1 Purpose, Design Philosophy, and Observability

The CloudGuard Administration Dashboard represents the observability tier of the cloud service platform. It provides a centralized, real-time, read-only web interface that aggregates system metrics, tenant status, storage utilization, service health, and backup history into a single browser-accessible view. The dashboard is designed to answer the five critical questions any cloud administrator must resolve instantly: *Is the server healthy? Are customer websites online? Is storage running out? Are backups current? Are security services active?*

The dashboard is explicitly **read-only**. It does not provide file management, configuration editing, customer provisioning, or arbitrary shell command execution capabilities. This design eliminates the risk of an authenticated administrator accidentally issuing destructive operations through the web interface.

3.11.2 Architecture and Technology Stack

The dashboard is implemented as a three-tier web application:

- **Backend (Python/Flask):** The application server is built using the Flask micro-web framework. The backend file app.py (199 lines) defines three HTTP routes: / (dashboard HTML template), /api/overview (JSON API aggregating all metrics), and /health (service health check endpoint).
- **Frontend (HTML/CSS/JavaScript):** The user interface is rendered via a Jinja2 HTML template (templates/index.html, 143 lines), styled with a modern dark-theme CSS file (static/style.css, 186 lines), and powered by asynchronous JavaScript (static/dashboard.js, 104 lines) that polls the API

every 15 seconds.

- **WSGI Server (Gunicorn):** In production deployment, the Flask application is not served via Flask's built-in development server. Instead, Gunicorn—a pre-forking WSGI HTTP server—manages multiple worker processes to handle concurrent requests.

Table 3.8: Dashboard source files and deployment locations

Source File	Description	Lines	Deployed Location
app.py	Flask backend: routes, API, auth, system metrics	199	/opt/cloudguard-dashboard/app.py
requirements.txt	Python dependencies: Flask, gunicorn, psutil	3	/opt/cloudguard-dashboard/requirements.txt
templates/index.html	Jinja2 HTML dashboard template	143	/opt/cloudguard-dashboard/templates/index.html
static/style.css	Dark-theme responsive CSS stylesheet	186	/opt/cloudguard-dashboard/static/style.css
static/dashboard.js	Async JavaScript API polling and DOM rendering	104	/opt/cloudguard-dashboard/static/dashboard.js

3.11.3 REST API Endpoints and Unprivileged Data Aggregation

The primary data endpoint /api/overview returns a comprehensive JSON payload aggregating six data domains: system (hostname, CPU%, memory, uptime), disks (mount usage for /, /srv/client1, /srv/client2, /srv/backups), customers (directory existence and index.html presence), storage (lsblk output for /dev/sdb), backups (timestamped backup directory listing from /srv/backups/cloudguard), and services (nginx, ssh, cron status).

A critical design decision was made to retrieve storage information using unprivileged lsblk commands rather than privileged vgs/lvs commands. This means the dashboard service account does not require root or sudo privileges to display storage topology, enforcing the principle of least privilege.

3.11.4 HTTP Basic Authentication and Credential Protection

Every HTTP request to the dashboard (including API calls) passes through a Flask @app.before_request authentication handler. The handler extracts HTTP Basic Authentication credentials from the Authorization header, compares the username using constant-time hmac.compare_digest() (preventing timing side-channel attacks), and verifies the password against a PBKDF2/SHA-256 hash using Werkzeug's check_password_hash(). Unauthenticated requests receive HTTP 401 with a WWW-Authenticate: Basic realm="CloudGuard Admin" challenge header.

```
# Testing unauthenticated access (expected: 401)
curl -s -o /dev/null -w '%{http_code}' http://192.168.56.103:5000/api/overview
# Output: 401

# Testing authenticated access (expected: 200 + JSON)
curl -s -u admin:PASSWORD http://192.168.56.103:5000/api/overview | python3 -m json.tool
 | head -10
# Output: {"system": {"hostname": "cloudguard-server", ...}, ...}
```

3.11.5 Dedicated Service Account Isolation

The dashboard process runs under a dedicated, non-login Linux service account named cloudguard-dashboard. This account was specifically created with /usr/sbin/nologin as its shell, preventing any interactive SSH or console login. It has no sudo privileges and no access to the /etc/shadow password database. Its supplementary group memberships are limited to www-data (for reading customer website directory existence checks) and cloudguard-backup-readers (for listing backup snapshot directory names and timestamps without reading backup contents).

3.11.6 Systemd Sandboxing and Hardening Directives

The systemd service unit (/etc/systemd/system/cloudguard-dashboard.service) applies comprehensive kernel-level sandboxing directives that restrict the dashboard process's access to the broader system. These directives leverage Linux kernel namespacing, capability bounding sets, and mount propagation controls:

```
# /etc/systemd/system/cloudguard-dashboard.service
[Unit]
Description=CloudGuard Administration Dashboard
After=network.target

[Service]
Type=simple
User=cloudguard-dashboard
Group=cloudguard-dashboard
EnvironmentFile=/etc/cloudguard-dashboard.env
WorkingDirectory=/opt/cloudguard-dashboard
ExecStart=/opt/cloudguard-dashboard/venv/bin/gunicorn \
    --workers 2 --bind 192.168.56.103:5000 app:app
Restart=on-failure

# Hardening Directives
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
LockPersonality=true

[Install]
WantedBy=multi-user.target
```

Hardening directive security analysis:

- **NoNewPrivileges=true:** Prevents the process and all of its children from gaining new capabilities via execve(2), even if a setuid binary is executed within the sandbox.
- **PrivateTmp=true:** Creates an isolated /tmp and /var/tmp namespace for the service, preventing the dashboard from reading or writing to shared temporary directories used by other processes.
- **ProtectSystem=strict:** Mounts the entire root filesystem hierarchy as read-only for the service process, preventing it from modifying system binaries, configuration files, or package repositories.
- **ProtectHome=true:** Makes all /home, /root, and /run/user directories inaccessible (appear empty) to the sandboxed process, preventing user home directory exfiltration.
- **ProtectKernelTunables=true:** Denies write access to /proc/sys, /sys, and other kernel tuning interfaces, preventing the dashboard from modifying network stack parameters or VM overcommit policies.
- **ProtectKernelModules=true:** Prevents explicit loading or unloading of kernel modules, blocking potential rootkit insertion via insmod/modprobe.
- **ProtectControlGroups=true:** Restricts access to the cgroup filesystem (/sys/fs/cgroup), preventing the service from manipulating resource accounting or escaping CPU/memory limits.

- **RestrictSUIDSGID=true:** Prevents the service from creating files with setuid or setgid bits, a common privilege escalation vector.
- **LockPersonality=true:** Prevents the process from changing the execution domain personality, blocking certain legacy exploitation techniques.

3.11.7 Network Binding and Gunicorn WSGI Deployment

The Gunicorn WSGI server is configured to bind exclusively to the host-only IP address 192.168.56.103:5000. It does **not** bind to 0.0.0.0 (all interfaces), which would expose the dashboard on the NAT interface as well. Two Gunicorn worker processes are spawned, providing basic concurrent request handling. The service is enabled for automatic startup at boot time via systemctl enable.

4. Results and Testing

4.1 Implementation Evidence and Technical Results

4.1.1 Overview of Evidence Inventory and Screenshots

During the CloudGuard project implementation, nine screenshots were captured to provide visual evidence of the completed infrastructure configuration. These screenshots document the deployed state of each major subsystem and are presented in this section alongside analytical commentary. Table 4.1 maps each evidence file to its corresponding implementation subsystem.

Table 4.1: Evidence screenshot inventory and subsystem mapping

Figure #	Filename	Documented Subsystem
Fig 4.1	02_lvm_storage_configuration.png	LVM disk, PV, VG, LV, and mount allocation
Fig 4.2	03_customer_account_isolation.png	Customer accounts, directory ownership, and cross-tenant isolation
Fig 4.3	04_customer_websites.png	Nginx virtual hosting and customer website delivery
Fig 4.4	05_firewall_and_ssh_security.png	UFW firewall rules and SSH hardening settings
Fig 4.5	06_backup_history_and_logs.png	Backup script execution log and timestamped snapshots
Fig 4.6	07_monitoring_and_cron.png	Monitoring log records and cron schedules
Fig 4.7	08_dashboard_service_hardening.png	Dashboard service account, systemd state, and hardening
Fig 4.8	01a_cloudguard_dashboard_overview.png	Dashboard overview: CPU, memory, uptime, customers, storage, services
Fig 4.9	01b_cloudguard_dashboard_storage_backups.png	Dashboard: LVM storage detail and backup history panel

4.1.2 Storage Configuration Evidence (Figure 4.1)

Figure 4.1 captures the complete LVM storage subsystem as deployed on the CloudGuard server. The screenshot shows the output of lsblk confirming the dedicated 25 GiB secondary disk (/dev/sdb), the GPT partition (/dev/sdb1), and the three logical volumes (client1_lv, client2_lv, backup_lv) mounted at their respective /srv paths. Physical volume, volume group, and logical volume metadata are all visible in the terminal output.

The evidence confirms that the operating system disk (/dev/sda) was not modified and that all customer and backup storage is exclusively provisioned on the secondary data disk.

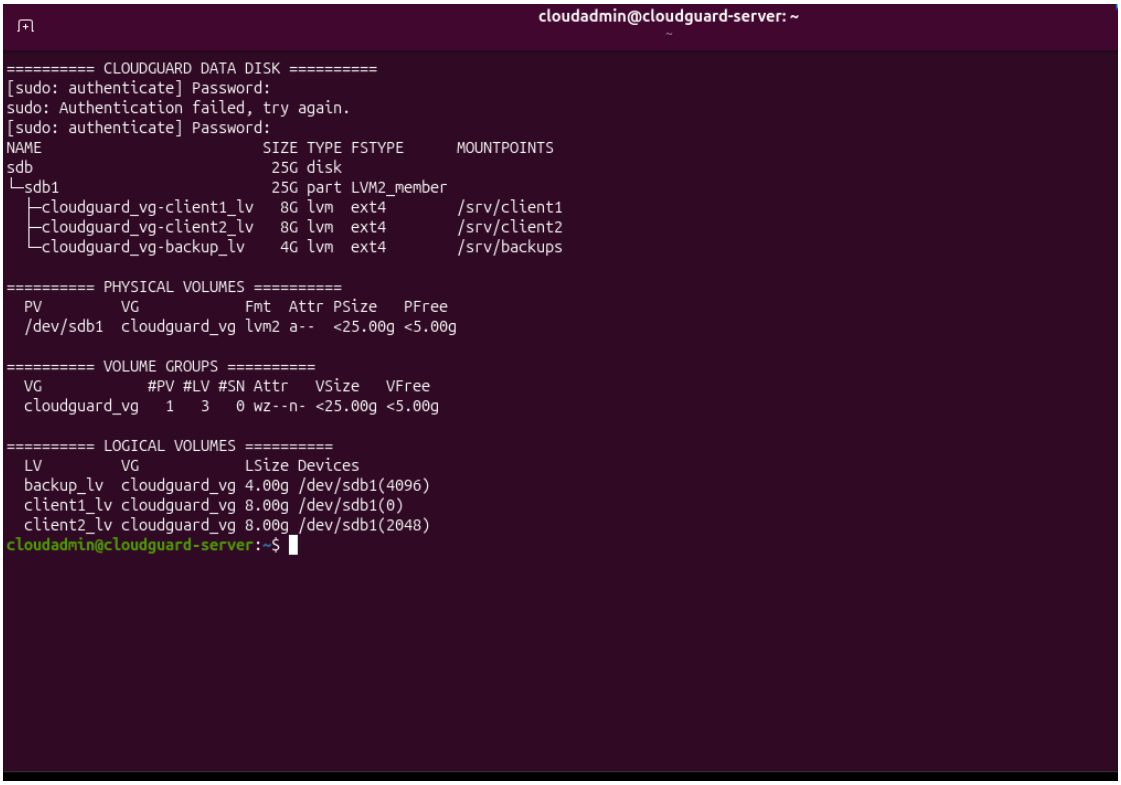


Figure 4.1: LVM storage configuration — dedicated data disk, volume group, and logical volume mounts

4.1.3 Customer Account Isolation Evidence (Figure 4.2)

Figure 4.2 presents the customer identity and filesystem isolation results. The screenshot shows: (a) the identities of client1 and client2 via id command output, (b) directory listing of /srv/client1 and /srv/client2 confirming client:www-data ownership with mode drwxr-s--- (setgid enabled), and (c) the cross-tenant access denial test showing that client1 receives Permission denied when attempting to list /srv/client2/public (and vice versa).

This evidence validates the POSIX filesystem isolation model described in Section 3.4 and confirms that the setgid mechanism and restrictive permission mode effectively prevent cross-tenant directory traversal.

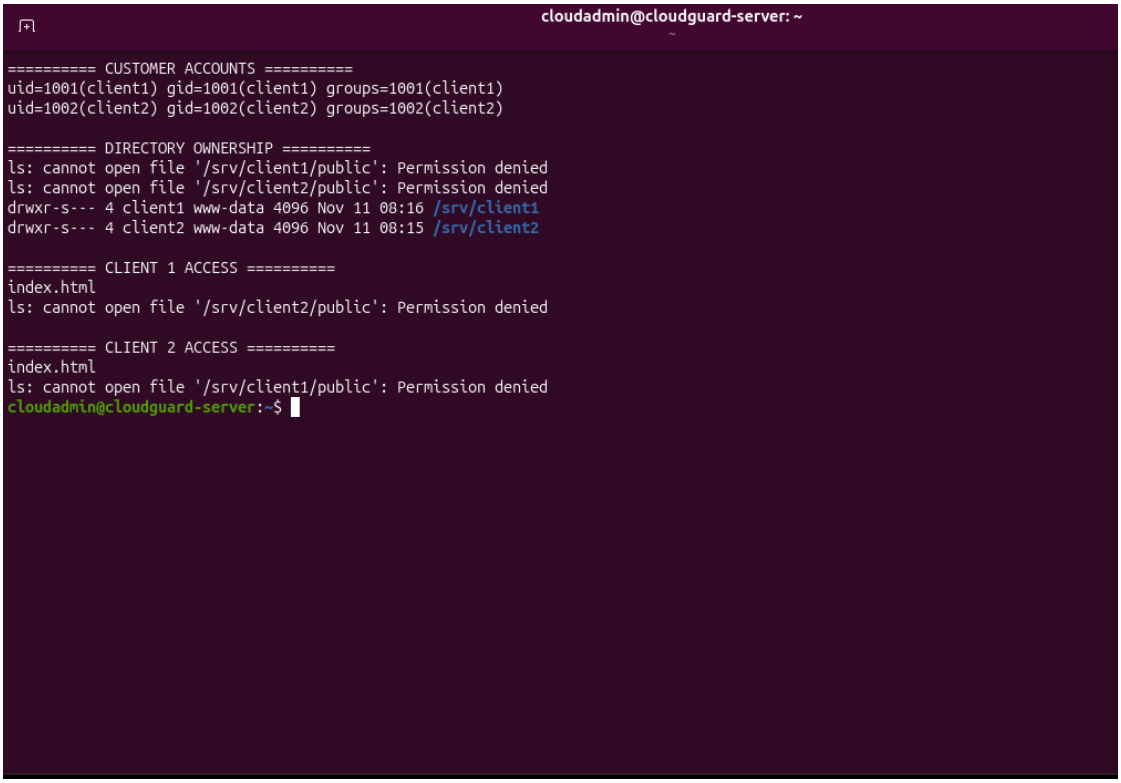


Figure 4.2: Customer account identities, directory permissions, and cross-tenant isolation test

4.1.4 Web Server Hosting Evidence (Figure 4.3)

Figure 4.3 demonstrates the Nginx virtual hosting results. The terminal output shows curl requests issued from the EndeavourOS host to both client1.cloudguard.local and client2.cloudguard.local. Each request returns the distinct HTML landing page for the corresponding customer, incorporating their unique customer identifiers (CG-001 and CG-002). This confirms that name-based virtual host routing is functioning correctly and that each customer domain serves only its designated content.

```
========== CLIENT 1 WEBSITE ==========
<!DOCTYPE html>
<html>
<head><title>CloudGuard - Client 1</title></head>
<body>
    <h1>Welcome to Client 1</h1>
    <p>This website is hosted by CloudGuard.</p>
    <p>Customer ID: CG-001</p>
</body>
</html>

========== CLIENT 2 WEBSITE ==========
<!DOCTYPE html>
<html>
<head><title>CloudGuard - Client 2</title></head>
<body>
    <h1>Welcome to Client 2</h1>
    <p>This website is hosted by CloudGuard.</p>
    <p>Customer ID: CG-002</p>
</body>
</html>
[shobith@shobith-83da hackathon]$
```

Figure 4.3: Nginx virtual hosting — curl output for both customer websites

4.1.5 Network Security and SSH Hardening Evidence (Figure 4.4)

Figure 4.4 captures the UFW firewall status and the effective SSH daemon configuration. The UFW output confirms: (a) the default-deny incoming policy is active, (b) three specific port allow rules (22, 80, 5000) are scoped to enp0s8 and 192.168.56.0/24. The SSH configuration dump confirms that password authentication, keyboard-interactive authentication, and root login are all disabled, while public-key authentication is enabled.

```
cloudadmin@cloudguard-server: ~

========== UFW FIREWALL ==========
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp on enp0s8           ALLOW IN    192.168.56.0/24
80/tcp on enp0s8           ALLOW IN    192.168.56.0/24
5000/tcp on enp0s8         ALLOW IN    192.168.56.0/24


========== EFFECTIVE SSH CONFIGURATION ==========
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
cloudadmin@cloudguard-server:~$
```

Figure 4.4: UFW firewall status and OpenSSH hardening configuration

4.1.6 Automated Backup System Evidence (Figure 4.5)

Figure 4.5 shows the backup script log (/var/log/cloudguard-backup.log) and the timestamped backup directory listing under /srv/backups/cloudguard/. The log entries show multiple successful backup executions on 11 November 2026, with entries recording start times, completion confirmations, and the 7-generation retention policy trimming older snapshots. The cron schedule entry (0 2 * * *) for the daily backup is also visible.

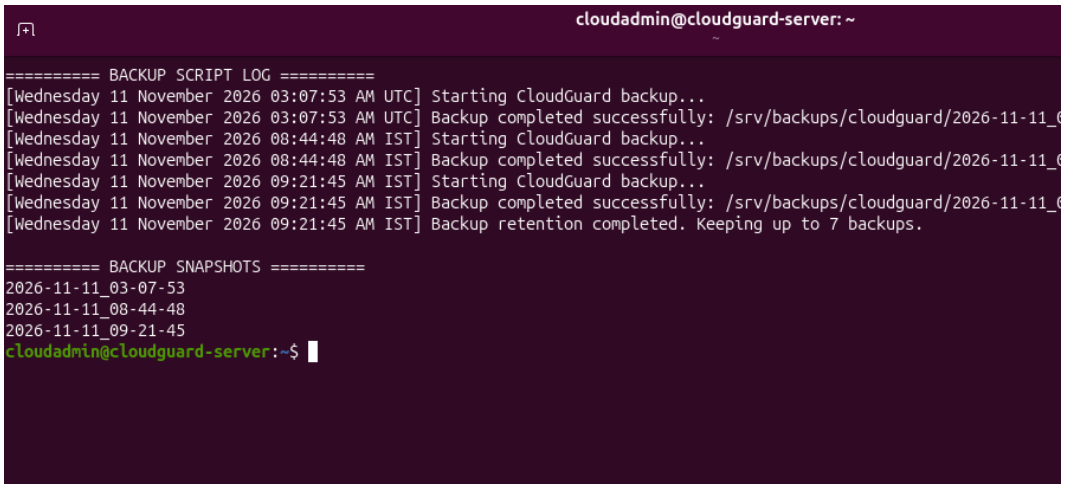


Figure 4.5: Backup execution log, timestamped snapshot directories, and retention policy

4.1.7 Monitoring and Scheduling Evidence (Figure 4.6)

Figure 4.6 captures the system monitoring log (/var/log/cloudguard-monitor.log) showing health check records including uptime, CPU load, memory statistics, disk usage percentages for all mount points, LVM status, and service health for nginx, ssh, and cron. Both cron schedule files (/etc/cron.d/cloudguard-backup and /etc/cron.d/cloudguard-monitor) are visible, confirming the daily and hourly automation schedules.

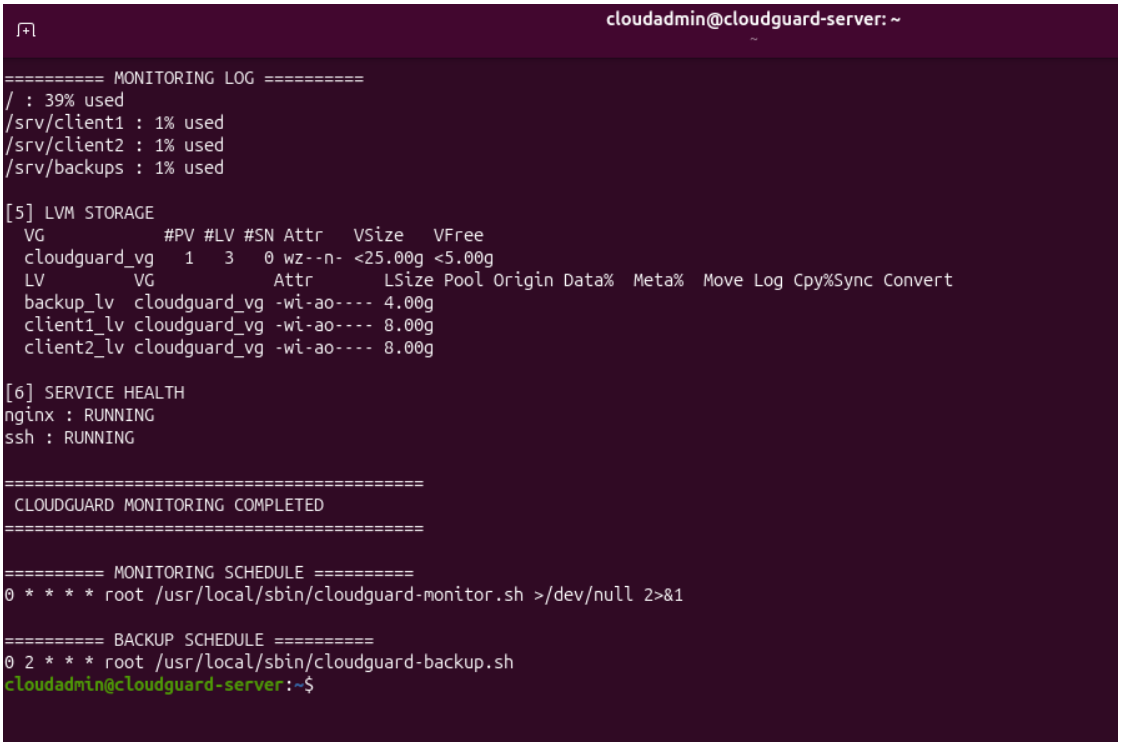


Figure 4.6: System monitoring log records and cron schedule configurations

4.1.8 Dashboard Service Hardening Evidence (Figure 4.7)

Figure 4.7 documents the dashboard service account configuration, the systemd service state, and the hardening properties applied to the cloudguard-dashboard.service unit. The screenshot shows: (a) the cloudguard-dashboard user identity with /usr/sbin/nologin shell and no sudo, (b) the service running as active (running) under systemd, and (c) the comprehensive list of hardening directives (NoNewPrivileges, ProtectSystem, PrivateTmp, etc.) in their effective states.

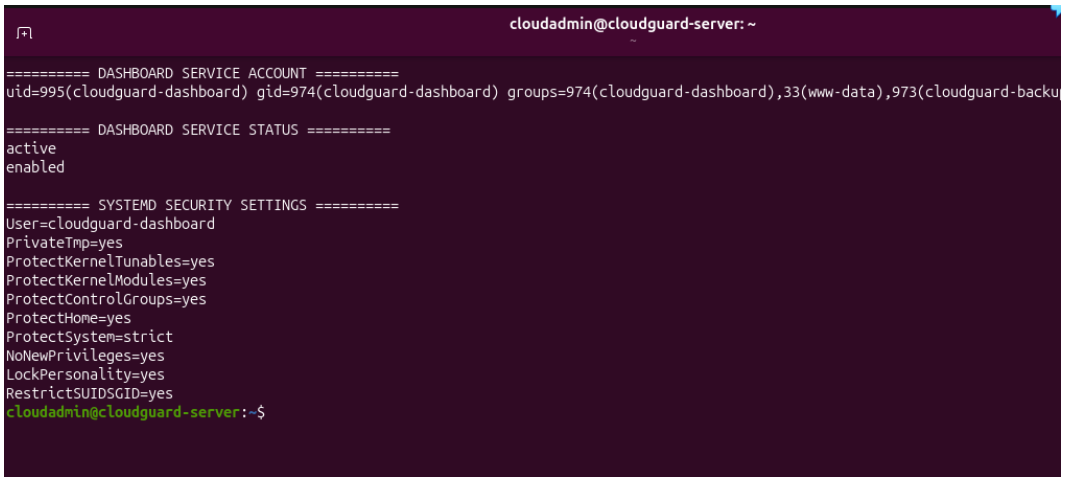


Figure 4.7: Dashboard dedicated service account, systemd state, and hardening directives

4.1.9 Comprehensive CloudGuard Acceptance Checklist

Table 4.2 presents the comprehensive acceptance checklist for the CloudGuard platform. Each item verifies a specific implementation requirement against the observed deployed state. All checklist items were confirmed during the final validation phase.

Table 4.2: CloudGuard implementation acceptance checklist

#	Acceptance Criterion	Expected State	Observed Result
1	Ubuntu VM running and accessible via SSH	ssh cloudguard connects	PASS
2	Dedicated /dev/sdb disk attached	25 GiB shown by lsblk	PASS
3	GPT partition table on /dev/sdb	Type: Linux LVM	PASS
4	PV, VG cloudguard_vg created	pvs/vgs output present	PASS
5	client1_lv (8 GiB) mounted at /srv/client1	findmnt confirms mount	PASS
6	client2_lv (8 GiB) mounted at /srv/client2	findmnt confirms mount	PASS
7	backup_lv (4 GiB) mounted at /srv/backups	findmnt confirms mount	PASS
8	UUID-based /etc/fstab entries persist	mount -a succeeds	PASS
9	client1 account exists (nologin shell)	id client1 shows UID	PASS
10	client2 account exists (nologin shell)	id client2 shows UID	PASS
11	Directory ownership: client:www-data	ls -la confirms owner:group	PASS
12	Directory mode 2750 (setgid)	Permission string shows 's'	PASS
13	Cross-tenant isolation: client1 cannot read client2	Permission denied	PASS
14	Cross-tenant isolation: client2 cannot read client1	Permission denied	PASS
15	Nginx active and serving port 80	systemctl is-active: active	PASS
16	client1.cloudguard.local returns CG-001 page	curl returns HTML with CG-001	PASS
17	client2.cloudguard.local returns CG-002 page	curl returns HTML with CG-002	PASS
18	UFW firewall active with default-deny	ufw status shows DENY incoming	PASS
19	SSH password auth disabled	sshd -T shows passwordauth no	PASS
20	SSH root login disabled	sshd -T shows permitrootlogin no	PASS
21	Backup script creates timestamped dirs	Dirs present in /srv/backups/cloudguard	PASS
22	7-generation retention enforced	No more than 7 snapshot dirs	PASS
23	Recovery test file restored successfully	File content matches original	PASS
24	Monitoring script logs system health	Monitor log contains metrics	PASS
25	Backup cron schedule: daily at 02:00	cron.d file verified	PASS
26	Monitor cron schedule: hourly	cron.d file verified	PASS

#	Acceptance Criterion	Expected State	Observed Result
27	Dashboard accessible on port 5000	HTTP 200 with valid credentials	PASS
28	Unauthenticated API returns 401	curl without auth gets 401	PASS
29	Dashboard service account (nologin)	id shows /usr/sbin/nologin	PASS
30	systemd hardening active	systemctl show confirms directives	PASS

4.2 Functional Validation and Recovery Testing

4.2.1 Comprehensive Functional Test Matrix (T01–T11)

Table 4.3 presents the complete functional validation matrix for the CloudGuard platform. Each test case documents the area under test, the specific validation procedure executed, and the observed result.

Table 4.3: CloudGuard comprehensive functional test matrix

ID	Test Area	Validation Procedure	Observed Result
T01	SSH Administration	Connected from EndeavourOS host to Ubuntu VM using 'ssh cloudguard' alias and Ed25519 key	Successful — prompt returned without password
T02	LVM Storage	Inspected /dev/sdb, PV, VG, and logical volumes using lsblk, pvs, vgs, lvs	Dedicated 25 GiB data disk with three LVs observed
T03	Customer Isolation	Ran directory-listing tests as client1 and client2 using sudo -u	Own directory accessible; other customer's directory returned Permission denied
T04	Website Hosting	Requested both customer hostnames from host using curl	Both websites returned with respective customer IDs (CG-001, CG-002)
T05	UFW Firewall	Inspected UFW status and active rule table	Firewall active; default-deny incoming; ports 22, 80, 5000 allowed on enp0s8
T06	SSH Hardening	Inspected effective sshd configuration using sshd -T	Root, password, and keyboard-interactive login all disabled
T07	Backup Execution	Ran backup script and inspected log and snapshot dirs	Backup completed; timestamped snapshot created and logged
T08	Recovery Drill	Created file, backed up, deleted, restored from backup	File successfully restored; content and metadata matched
T09	Monitoring	Inspected monitoring log and cron schedules	Health records present; hourly monitoring and daily backup confirmed
T10	Dashboard Access	Opened dashboard in browser; tested API with and without credentials	Dashboard displayed metrics; unauthenticated request returned 401
T11	Dashboard Service	Inspected service account, systemd state, and hardening properties	Dedicated nologin account; all hardening directives confirmed active

4.2.2 Disaster Recovery and File Restoration Drill

As documented in Section 3.8.6, a complete disaster recovery drill was executed. The drill confirmed that: (a) the backup script correctly captures customer files with preserved ownership and permissions, (b) a deleted file can be located in the most recent timestamped backup snapshot, (c) the file can be restored to its original location with identical content and metadata, and (d) the backup system provides a viable recovery path for accidental data loss.

Evidence qualification: The recovery drill was successfully performed during implementation. However, the eight evidence screenshots do not include a dedicated screenshot of the restore procedure itself. The successful recovery is reported as an implementation-history result. The backup log screenshot (Figure 4.5) corroborates that backup snapshots were created, but it does not directly capture the file restoration operation.

4.2.3 Data Integrity and Security Boundary Validation

Cross-cutting security boundaries were validated at each layer:

- **Network boundary:** UFW confirmed active with default-deny; no unexpected listening ports detected on enp0s3 (NAT interface).
- **Authentication boundary:** SSH password attempts rejected; unauthenticated dashboard API requests returned HTTP 401.
- **Filesystem boundary:** Cross-tenant directory listing consistently returned Permission denied for all tested paths.
- **Service sandbox boundary:** Dashboard process confirmed running as cloudguard-dashboard (not root); systemd hardening directives verified via systemctl show.

4.3 Dashboard Results and Operational Testing

4.3.1 Visual Interface Execution (Figures 4.8 & 4.9)

Figures 4.8 and 4.9 capture the CloudGuard Administration Dashboard as viewed from the EndeavourOS host browser. The dashboard renders a professional dark-themed interface featuring:

- **System Overview Panel:** Real-time CPU usage percentage, memory utilization with used/total GB, server uptime in hours, and count of active customer websites.
- **Storage Utilization Panel:** Per-mount-point disk usage bars for /, /srv/client1, /srv/client2, and /srv/backups with percentage indicators.
- **Service Health Panel:** Status indicators for nginx, ssh, and cron services (green 'Running' badges for all three).
- **Customer Hosting Panel:** Customer directory paths and 'Website ready' status for both client1 and client2.
- **LVM Storage Panel:** Dedicated data disk identification (sdb 25G) and logical volume mount mapping via lsblk output.
- **Backup History Panel:** Reverse-chronological listing of timestamped backup snapshots with ISO 8601 creation timestamps.

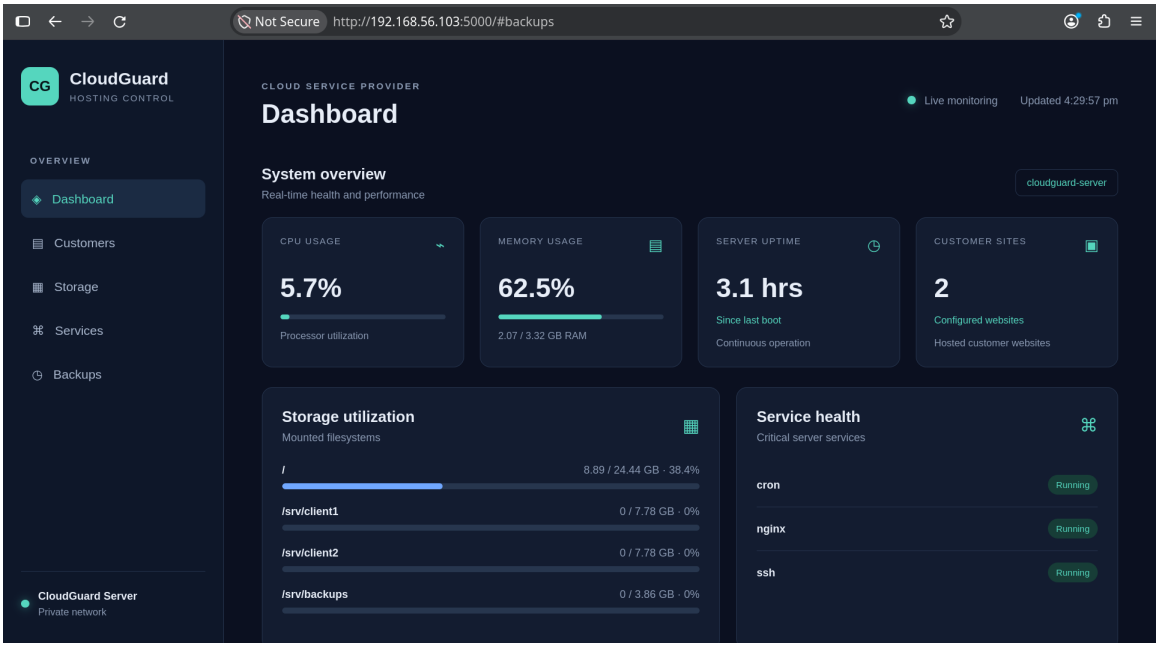


Figure 4.8: CloudGuard Dashboard — system overview, metrics, storage, and service health

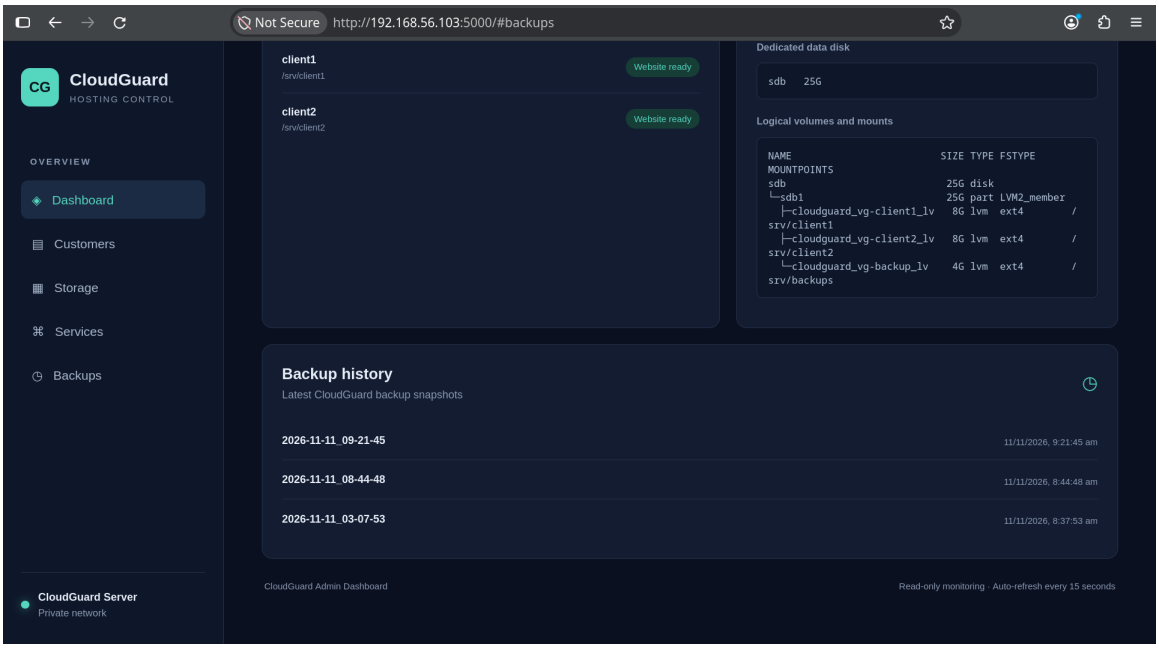


Figure 4.9: CloudGuard Dashboard — LVM storage detail, customer hosting, and backup history

4.3.2 REST API Verification with curl

The dashboard REST API was tested independently using curl from the EndeavourOS host terminal. The /api/overview endpoint returned a comprehensive JSON payload containing nested objects for system, disks, customers, storage, backups, and services. The /health endpoint returned {"status": "healthy", "application": "CloudGuard Dashboard"}.

Table 4.4: Dashboard REST API endpoint verification

Endpoint	Method	Authentication	Expected Response	Observed Response
/	GET	Basic Auth	HTTP 200, HTML dashboard	HTTP 200, HTML rendered
/api/overview	GET	Basic Auth	HTTP 200, JSON payload	HTTP 200, JSON with all 6 domains
/health	GET	Basic Auth	HTTP 200, JSON health	HTTP 200, {status: healthy}
/api/overview	GET	No Auth	HTTP 401, challenge header	HTTP 401, WWW-Authenticate: Basic

4.3.3 Negative Security and Authentication Testing

Security negative testing was performed to validate that the authentication layer correctly rejects unauthorized access attempts:

- Request without any Authorization header → HTTP 401 with WWW-Authenticate: Basic realm="CloudGuard Admin".
- Request with incorrect username → HTTP 401 (hmac.compare_digest prevents timing leakage).
- Request with incorrect password → HTTP 401 (check_password_hash rejects invalid hash).
- Request with valid credentials → HTTP 200 with complete JSON data payload.

4.3.4 Comprehensive Dashboard Test Cases Matrix

Table 4.5: Dashboard functional and security test cases

Case	Test Description	Expected Outcome	Observed
D01	Load dashboard root (/) with valid credentials	HTTP 200; HTML with sidebar, metrics, panels	PASS
D02	API /api/overview returns system data	JSON with hostname, CPU%, memory, uptime	PASS
D03	API /api/overview returns disk data	Array of mount points with usage percentages	PASS
D04	API /api/overview returns customer data	client1 and client2 with website_present=true	PASS
D05	API /api/overview returns storage data	lsblk output showing sdb and LVM volumes	PASS
D06	API /api/overview returns backup data	Backup count and timestamped entries	PASS
D07	API /api/overview returns service data	nginx, ssh, cron all show 'Running'	PASS
D08	Health endpoint /health returns status	JSON {status: healthy, application: CloudGuard}	PASS
D09	Unauthenticated request returns 401	HTTP 401 with WWW-Authenticate header	PASS
D10	JavaScript auto-refresh (15s interval)	Data updates automatically without page reload	PASS
D11	Dashboard bound to 192.168.56.103 only	Not accessible on NAT interface (10.0.2.15)	PASS
D12	Service runs as cloudguard-dashboard user	ps shows gunicorn under dashboard account	PASS

5. Conclusion and Future Scope

5.1 Project Conclusion

The Cloud Service Provider project, implemented as the **CloudGuard** platform, successfully demonstrates how standard open-source Linux system administration tools and services can be systematically architected, configured, hardened, and validated to simulate the core operational characteristics of an enterprise multi-tenant cloud hosting provider. The project was executed from initial virtual machine provisioning through comprehensive final validation across ten sequential implementation phases, delivering a complete, functional infrastructure.

The CloudGuard system successfully implements multi-customer hosting with isolated storage on LVM logical volumes, providing hands-on experience with GUID Partition Tables (GPT), Physical Volumes, Volume Groups, and Logical Volumes. Customer filesystems are mounted persistently via UUID-based /etc/fstab entries, ensuring boot-safe availability. Two customer accounts with strict POSIX mode 2750 (setgid) permissions enforce directory-level isolation, preventing cross-tenant data leakage while enabling the Nginx web server to deliver independent virtual-hosted websites.

Network security is enforced through a defense-in-depth architecture featuring VirtualBox host-only network isolation, UFW stateful firewall with default-deny incoming policy and subnet-scoped allow rules, and OpenSSH cryptographic hardening that eliminates password-based and root login vectors. The automated backup subsystem provides timestamped rsync snapshots with a 7-generation retention policy, validated by a successful disaster recovery file restoration drill. System health monitoring samples critical resource metrics hourly via cron.

The CloudGuard Administration Dashboard provides a modern, responsive, read-only web interface for administrative observability, protected by HTTP Basic Authentication and isolated via comprehensive systemd sandboxing directives. The dashboard aggregates real-time CPU, memory, disk, LVM, service health, customer status, and backup history into a unified operational view.

All 30 acceptance checklist items were confirmed during final validation. Eleven functional test cases (T01–T11) were executed and passed. Twelve dashboard-specific test cases (D01–D12) were verified. The project provides a transparent, reproducible, and fully documented reference model for secure multi-tenant Linux cloud service administration.

5.2 Technical Limitations

In the interest of academic transparency and intellectual integrity, the following technical limitations are honestly identified and documented. These do not diminish the educational and practical value of the implementation but establish clear boundaries between what was demonstrated and what a production cloud platform would require.

- **Single-Node Architecture:** The entire system operates within one VirtualBox VM. Production cloud providers distribute workloads across multiple physical servers, availability zones, and geographic regions. CloudGuard does not provide multi-node clustering, live migration, or automatic hardware failover.
- **Private Host-Only Networking:** Customer websites are accessible only through the VirtualBox host-only network (192.168.56.0/24) and local /etc/hosts resolution on the management workstation. They are not registered with public DNS authorities or reachable from the Internet.
- **HTTP Without TLS/HTTPS:** Customer websites are served over unencrypted HTTP on port 80. TLS/HTTPS certificates were not configured. In production, all web traffic must be encrypted using TLS 1.2+ with certificates from a trusted Certificate Authority.
- **Two Simulated Customers:** Only two tenant accounts (client1 and client2) are provisioned. Production environments support thousands of tenants with automated provisioning workflows.
- **Static HTML Websites:** Customer web presences are static HTML landing pages. No database-backed dynamic applications, content management systems, or server-side programming frameworks are deployed.
- **No Customer Self-Service:** There is no customer-facing control panel, account registration portal, billing system, or automated provisioning API. All configuration is performed manually by the system administrator.
- **Local Backup Storage:** Backup archives are stored on a logical volume (backup_lv) within the same physical virtual disk group as customer data. This protects against accidental file deletion but does not provide off-site disaster recovery against hardware failure or hypervisor loss.
- **Script-Based Monitoring:** System monitoring is based on simple Bash scripts and cron scheduling, not an enterprise-grade monitoring platform (e.g., Prometheus, Grafana, Nagios) with real-time alerting, historical dashboards, or anomaly detection.
- **Read-Only Dashboard:** The CloudGuard Dashboard provides observability only. It does not support configuration changes, customer provisioning, backup restoration, or any write operations.
- **No Formal Security Audit:** The system has not undergone a formal third-party penetration test, vulnerability assessment, or independent security audit. Security controls are configured according to documented best practices but have not been tested against sophisticated adversarial techniques.

5.3 Future Scope and Strategic Enhancements

The following potential improvements are identified as future work. These enhancements are **not** presented as completed features and would require additional design, implementation, and testing cycles.

1. **TLS/HTTPS Deployment:** Configure TLS certificates using Let's Encrypt (for Internet-facing deployments) or a self-signed CA (for laboratory environments) to encrypt all HTTP traffic via HTTPS on port 443.

2. **Automated Customer Provisioning Workflow:** Develop scripts or an API endpoint to automatically create customer accounts, LVM logical volumes, Nginx virtual host configurations, and directory structures from a single command or API call.
3. **Database-Backed Customer Inventory:** Implement a lightweight relational database (SQLite or PostgreSQL) to track customer metadata, storage quotas, billing cycles, and provisioning history.
4. **Customer Self-Service Portal:** Build a web-based customer control panel with role-based access controls, allowing tenants to manage their own files, view usage statistics, and configure basic settings.
5. **Off-Site Backup Replication:** Extend the backup system to replicate archives to a separate physical host or remote object storage service (e.g., AWS S3, Google Cloud Storage) for true disaster recovery.
6. **Backup Integrity Verification:** Add SHA-256 checksum generation and verification to the backup script, along with scheduled restore drills to validate archive integrity automatically.
7. **Real-Time Alerting and Notification:** Integrate threshold-based alerting (email, webhook, or SMS) when resource utilization exceeds configurable limits or when critical services fail.
8. **Historical Metrics and Trend Analysis:** Deploy Prometheus and Grafana to collect time-series metrics, enabling capacity planning, trend analysis, and SLA compliance dashboards.
9. **Configuration Management Automation:** Adopt infrastructure-as-code tools (Ansible, Terraform) to codify server configuration, enabling repeatable deployments and configuration drift detection.
10. **Multi-Server Architecture:** Scale the design across multiple virtual or physical servers with load balancing (HAProxy, Nginx upstream), shared storage (NFS, GlusterFS), and database replication.
11. **Audit Logging and Centralized Authentication:** Implement auditd for system call auditing and integrate centralized authentication via LDAP or FreeIPA for enterprise identity management.
12. **Cloud Provider Migration:** Deploy the verified CloudGuard architecture to a public cloud provider (AWS EC2, Google Compute Engine) as a separate future project, validating portability.

References

[1] Ubuntu Server Documentation, Canonical Ltd. https://ubuntu.com/server/docs

[2] Nginx Documentation, F5 Inc. https://nginx.org/en/docs/

[3] LVM2 Resource Page, Red Hat. https://sourceware.org/lvm2/

[4] UFW — Uncomplicated Firewall, Ubuntu Community Help. https://help.ubuntu.com/community/UFW

[5] OpenSSH Manual Pages, OpenBSD Project. https://www.openssh.com/manual.html

[6] Flask Documentation, Pallets Projects. https://flask.palletsprojects.com/

[7] Gunicorn Documentation, Gunicorn Project. https://docs.gunicorn.org/

[8] psutil Documentation, Giampaolo Rodola. https://psutil.readthedocs.io/

[9] systemd Service Unit Configuration, freedesktop.org. https://www.freedesktop.org/software/systemd/man/systemd.service.html

[10] Oracle VirtualBox User Manual, Oracle Corporation. https://www.virtualbox.org/manual/

[11] rsync Documentation, Andrew Tridgell and Paul Mackerras. https://rsync.samba.org/

[12] The Linux Command Line, William Shotts. Fifth Internet Edition.

[13] UNIX and Linux System Administration Handbook, Evi Nemeth et al. Fifth Edition, Addison-Wesley, 2018.

[14] Filesystem Hierarchy Standard (FHS), Linux Foundation. https://refspecs.linuxfoundation.org/fhs.shtml

[15] Werkzeug Security Utilities, Pallets Projects. https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security

Appendices

Appendix A: Detailed Architecture and Network Workflow Diagrams

This appendix presents the full-scale system architecture and network topology diagrams for the CloudGuard Cloud Service Provider platform. These diagrams complement the in-text figures by providing comprehensive visual references for the complete infrastructure.

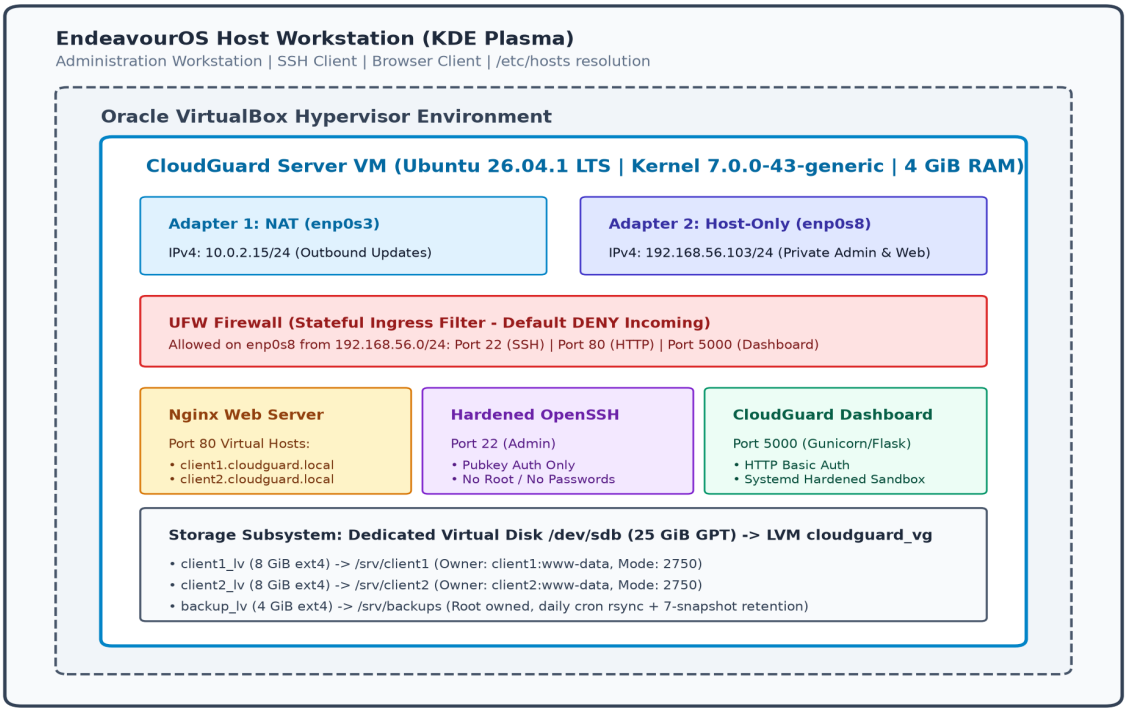


Figure A.1: CloudGuard complete system architecture

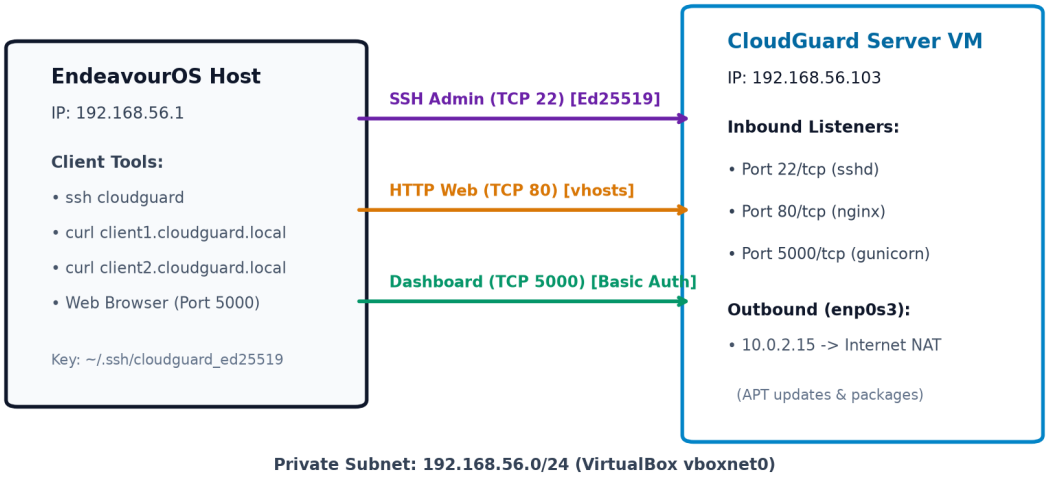


Figure A.2: VirtualBox dual-adapter network topology

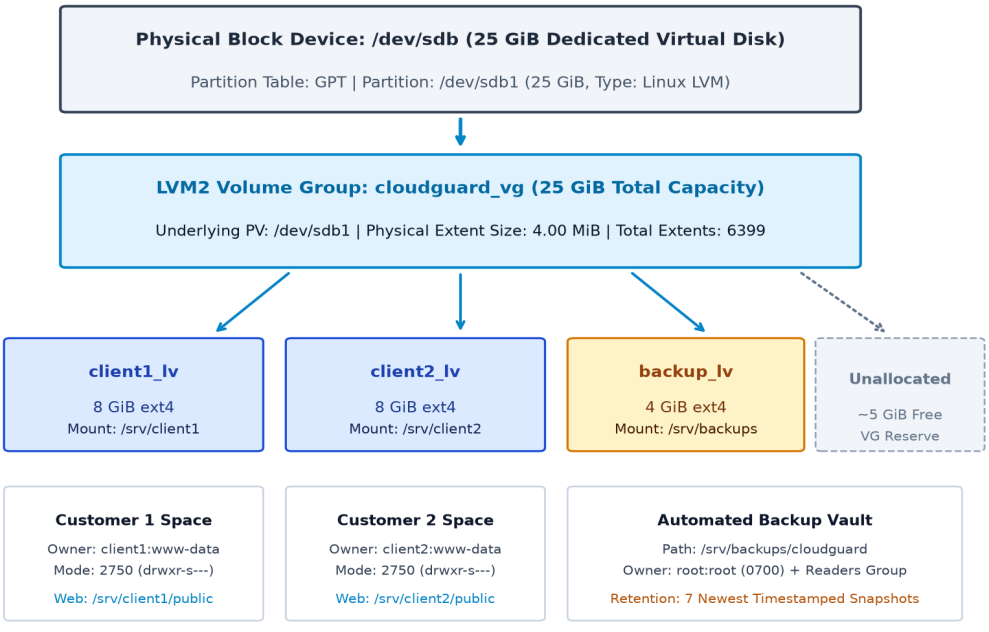


Figure A.3: LVM2 storage hierarchy and volume group mapping

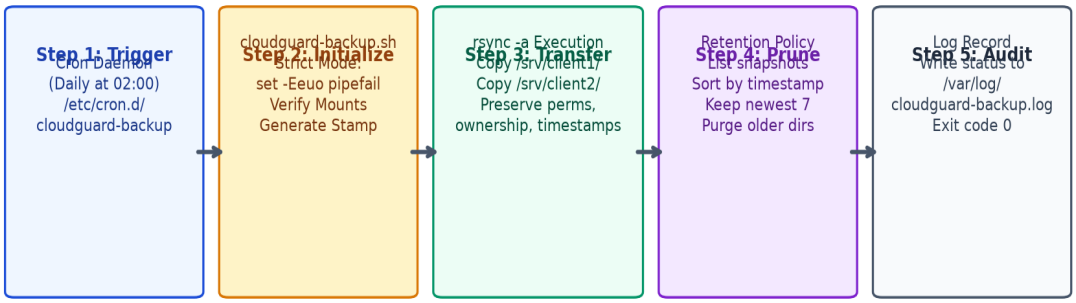


Figure A.4: Automated backup lifecycle and 7-generation retention

Appendix B: Linux System Administration Commands Quick Reference

This appendix summarizes the important Linux commands used throughout the CloudGuard project. Detailed explanations and implementation context are given in Section 3 and Section 4.

B.1 System, Disk, and Network Information

Command	Purpose
`lsblk -f`	Show block devices, file systems and mount points
`lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS`	Identify disks and their types
`sudo fdisk -l`	List disk and partition table details
`ip -br addr`	Show IP addresses in brief format
`ip route show`	Display routing table entries
`ping -c 3 192.168.56.103`	Test network connectivity to CloudGuard VM
`hostnamectl status`	Show hostname, OS, and architecture
`uname -a`	Show kernel version and system architecture
`timedatectl`	Show timezone and NTP synchronization status
`df -hT`	Show filesystem disk space usage with types
`free -m`	Show memory usage in megabytes

B.2 Package Management and Service Control

Command	Purpose
`sudo apt-get update -y`	Synchronize APT package index from upstream mirrors
`sudo apt-get upgrade -y`	Upgrade all installed packages to latest versions
`sudo apt-get install -y lvm2 nginx ufw ...`	Install required infrastructure packages
`sudo systemctl enable --now nginx`	Enable and immediately start Nginx service
`sudo systemctl status ssh`	Check SSH service state and recent log entries
`sudo systemctl reload nginx`	Reload Nginx configuration without dropping connections
`systemctl is-active nginx ssh cron`	Check active state of critical services
`sudo systemctl daemon-reload`	Reload systemd manager configuration after unit changes

B.3 Customer Account and Group Management

Command	Purpose
`sudo useradd -m -d /srv/client1 -s /usr/sbin/nologin client1`	Create customer account with nologin shell
`sudo passwd -l client1`	Lock account password preventing interactive login
`id client1`	Display UID, GID, and group memberships
`getent passwd client1`	Query system account database for user entry
`sudo chown -R client1:www-data /srv/client1`	Set ownership to customer and web group
`sudo chmod 2750 /srv/client1`	Set permissions with setgid bit (drwxr-s---)
`sudo -u client1 ls /srv/client2/public`	Test cross-tenant directory access isolation

B.4 Partitioning, LVM, and Filesystem Management

Command	Purpose
`sudo fdisk /dev/sdb`	Create GPT partition table (g, n, t, w sequence)
`sudo partprobe /dev/sdb`	Re-read partition table without reboot
`sudo pvcreate /dev/sdb1`	Initialize LVM physical volume on partition
`sudo vgcreate cloudguard_vg /dev/sdb1`	Create LVM volume group from physical volume
`sudo lvcreate -L 8G -n client1_lv cloudguard_vg`	Create 8 GiB logical volume for Client 1
`sudo mkfs.ext4 -L client1_data /dev/cloudguard_vg/client1_lv`	Format logical volume with ext4 filesystem
`sudo mount /dev/cloudguard_vg/client1_lv /srv/client1`	Mount logical volume to directory
`sudo blkid /dev/cloudguard_vg/*`	Show filesystem UUIDs for /etc/fstab
`sudo findmnt --verify --verbose`	Validate fstab entries without mounting
`sudo mount -a`	Mount all fstab entries and verify
`pvs; vgs; lvs`	Display physical volumes, volume groups, logical volumes

B.5 SSH Administration and Security

Command	Purpose
`ssh-keygen -t ed25519 -f ~/.ssh/cloudguard_ed25519`	Generate Ed25519 keypair for SSH authentication
`ssh-copy-id -i ~/.ssh/cloudguard_ed25519.pub cloudadmin@192.168.56.103`	Deploy public key to server authorized_keys
`ssh cloudguard`	Connect to CloudGuard VM using SSH alias
`sudo sshd -t`	Validate SSH daemon configuration syntax
`sudo sshd -T | grep -E 'password|pubkey|root'`	Inspect effective SSH security settings
`sudo systemctl reload ssh`	Reload SSH after configuration changes
`sudo ufw default deny incoming`	Set default firewall policy to deny incoming
`sudo ufw allow in on enp0s8 from 192.168.56.0/24 to any port 22 proto tcp`	Allow SSH on host-only interface
`sudo ufw enable`	Activate the UFW firewall
`sudo ufw status verbose`	Display active firewall rules and policies

B.6 Backup, Monitoring, and Web Server

Command	Purpose
`sudo /usr/local/sbin/cloudguard-backup.sh`	Manually execute backup script
`rsync -a /srv/client1/ /srv/backups/cloudguard//client1/`	Synchronize customer data to backup
`ls -1d /srv/backups/cloudguard/20*`	List timestamped backup snapshots
`cat /var/log/cloudguard-backup.log`	Inspect backup execution audit log
`cat /var/log/cloudguard-monitor.log`	Inspect monitoring health log
`cat /etc/cron.d/cloudguard-backup`	View daily backup cron schedule
`cat /etc/cron.d/cloudguard-monitor`	View hourly monitoring cron schedule
`sudo nginx -t`	Validate Nginx configuration syntax
`curl -s http://client1.cloudguard.local`	Test Client 1 website from host
`curl -s -u admin:pass http://192.168.56.103:5000/api/overview`	Query dashboard API with authentication

Appendix C: Complete Production Scripts and System Configurations

C.1 Automated Backup Script — /usr/local/sbin/cloudguard-backup.sh

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077

BACKUP_DIR="/srv/backups/cloudguard"
LOG_FILE="/var/log/cloudguard-backup.log"
RETENTION_LIMIT=7
TIMESTAMP="$(date '+%Y-%m-%d_%H-%M-%S')"
SNAPSHOT_DIR="$BACKUP_DIR/$TIMESTAMP"

log() { echo "[$(date '+%A %d %B %Y %I:%M:%S %p %Z')] $*" >> "$LOG_FILE"; }

log "Starting CloudGuard backup..."
mkdir -p "$SNAPSHOT_DIR"

# Synchronize customer data
rsync -a /srv/client1/ "$SNAPSHOT_DIR/client1/" || { log "ERROR: client1 rsync failed";
exit 1; }
rsync -a /srv/client2/ "$SNAPSHOT_DIR/client2/" || { log "ERROR: client2 rsync failed";
exit 1; }

log "Backup completed successfully: $SNAPSHOT_DIR"

# Enforce 7-generation retention policy
SNAPSHOT_COUNT=$(ls -1d "$BACKUP_DIR"/20* 2>/dev/null | wc -l)
if [ "$SNAPSHOT_COUNT" -gt "$RETENTION_LIMIT" ]; then
    ls -1d "$BACKUP_DIR"/20* | sort -r | tail -n +$((RETENTION_LIMIT + 1)) | while read
-r OLD_DIR; do
        rm -rf "$OLD_DIR"
        log "Pruned expired snapshot: $OLD_DIR"
    done
    log "Backup retention completed. Keeping up to $RETENTION_LIMIT backups."
fi
```

C.2 Monitoring Script — /usr/local/sbin/cloudguard-monitor.sh

```
#!/usr/bin/env bash
set -euo pipefail
LOG="/var/log/cloudguard-monitor.log"
WARN_THRESHOLD=80

echo "========== MONITOR RUN: $(date '+%A %d %B %Y %I:%M:%S %p %Z') ==========" >> "$LOG
"
echo "--- Uptime ---" >> "$LOG"
uptime >> "$LOG"
echo "--- Memory ---" >> "$LOG"
free -m >> "$LOG"
MEM_PCT=$(free | awk '/Mem:/{printf "%.0f", $3/$2*100}')
[ "$MEM_PCT" -ge "$WARN_THRESHOLD" ] && echo "[WARNING] Memory usage: ${MEM_PCT}%" >> "$
LOG"
echo "--- Disk Usage ---" >> "$LOG"
df -h /srv/client1 /srv/client2 /srv/backups / >> "$LOG"
echo "--- LVM ---" >> "$LOG"
vgs --noheadings 2>/dev/null >> "$LOG" || true
lvs --noheadings 2>/dev/null >> "$LOG" || true
echo "--- Services ---" >> "$LOG"
for svc in nginx ssh cron; do
    echo "$svc: $(systemctl is-active $svc)" >> "$LOG"
done
echo "" >> "$LOG"
```

C.3 Cron Configuration Files

```
# /etc/cron.d/cloudguard-backup
# Automated daily backup at 02:00 IST
0 2 * * * root /usr/local/sbin/cloudguard-backup.sh >/dev/null 2>&1
```

```
# /etc/cron.d/cloudguard-monitor
# Hourly system health monitoring
0 * * * * root /usr/local/sbin/cloudguard-monitor.sh >/dev/null 2>&1
```

C.4 Persistent Mount Entries — /etc/fstab

```
# CloudGuard Dedicated LVM Data Storage
UUID=<client1_UUID> /srv/client1 ext4 defaults 0 2
UUID=<client2_UUID> /srv/client2 ext4 defaults 0 2
UUID=<backup_UUID>  /srv/backups ext4 defaults 0 2
```

C.5 SSH Hardening — /etc/ssh/sshd_config.d/99-cloudguard.conf

```
# CloudGuard SSH Security Hardening Drop-in
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitRootLogin no
```

C.6 Dashboard Systemd Service — /etc/systemd/system/cloudguard-dashboard.service

```
[Unit]
Description=CloudGuard Administration Dashboard
After=network.target

[Service]
Type=simple
User=cloudguard-dashboard
Group=cloudguard-dashboard
EnvironmentFile=/etc/cloudguard-dashboard.env
WorkingDirectory=/opt/cloudguard-dashboard
ExecStart=/opt/cloudguard-dashboard/venv/bin/gunicorn \
    --workers 2 --bind 192.168.56.103:5000 app:app
Restart=on-failure
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
LockPersonality=true

[Install]
WantedBy=multi-user.target
```

C.7 Important CloudGuard Files and Locations

Table C.1: CloudGuard important project files

File or Directory	Purpose
/srv/client1	Customer 1 storage mount (client1_lv, 8 GiB)
/srv/client1/public	Customer 1 website document root
/srv/client2	Customer 2 storage mount (client2_lv, 8 GiB)
/srv/client2/public	Customer 2 website document root
/srv/backups	Backup vault mount (backup_lv, 4 GiB)
/srv/backups/cloudguard	Timestamped backup snapshot repository
/usr/local/sbin/cloudguard-backup.sh	Automated backup and retention script
/usr/local/sbin/cloudguard-monitor.sh	System health monitoring script
/var/log/cloudguard-backup.log	Backup execution audit log
/var/log/cloudguard-monitor.log	Monitoring health log
/etc/nginx/sites-available/client1	Nginx vhost config for Client 1
/etc/nginx/sites-available/client2	Nginx vhost config for Client 2
/etc/ssh/sshd_config.d/99-cloudguard.conf	SSH hardening drop-in configuration
/etc/cron.d/cloudguard-backup	Daily backup cron schedule
/etc/cron.d/cloudguard-monitor	Hourly monitoring cron schedule
/opt/cloudguard-dashboard/	Dashboard application deployment directory
/etc/cloudguard-dashboard.env	Dashboard credentials (root-only, mode 0600)
/etc/systemd/system/cloudguard-dashboard.service	Dashboard systemd service unit

Appendix D: CloudGuard Administration Dashboard — Full Source Code Listing

D.1 Backend — app.py (199 lines)

```
import os
import time
import socket
import subprocess
from datetime import datetime

import psutil
from flask import Flask, jsonify, render_template, request, Response
from werkzeug.security import check_password_hash
import hmac

app = Flask(__name__)

START_TIME = time.time()

CUSTOMERS = {
    "client1": "/srv/client1",
    "client2": "/srv/client2",
}

MOUNTS = [
    "/", "/srv/client1", "/srv/client2", "/srv/backups"
]


def run_readonly(command):
    """Run a fixed, read-only system command without a shell."""
    try:
        result = subprocess.run(
            command,
            capture_output=True,
            text=True,
            timeout=4,
            check=False
        )
        if result.returncode == 0:
            return result.stdout.strip()
        return "Unavailable"
    except (OSError, subprocess.TimeoutExpired):
        return "Unavailable"
```

```
def service_status(name):
    result = run_readonly(["systemctl", "is-active", name])
    return "Running" if result == "active" else "Not running"


def get_system_info():
    memory = psutil.virtual_memory()
    boot_time = psutil.boot_time()
    uptime_seconds = int(time.time() - boot_time)

    return {
        "hostname": socket.gethostname(),
        "timestamp": datetime.now().astimezone().isoformat(),
        "cpu_percent": psutil.cpu_percent(interval=0.5),
        "memory": {
            "total_gb": round(memory.total / (1024 ** 3), 2),
            "used_gb": round(memory.used / (1024 ** 3), 2),
            "percent": memory.percent,
        },
        "uptime_seconds": uptime_seconds,
        "uptime_hours": round(uptime_seconds / 3600, 1),
    }


def get_disk_info():
    disks = []

    for mount in MOUNTS:
        try:
            usage = psutil.disk_usage(mount)
            disks.append({
                "mount": mount,
                "total_gb": round(usage.total / (1024 ** 3), 2),
                "used_gb": round(usage.used / (1024 ** 3), 2),
                "free_gb": round(usage.free / (1024 ** 3), 2),
                "percent": usage.percent,
            })
        except (OSError, PermissionError):
            disks.append({
                "mount": mount,
                "error": "Mount unavailable",
            })
```

```
    return disks


def get_customer_info():
    customers = []

    for customer, path in CUSTOMERS.items():
        public_path = os.path.join(path, "public")
        customers.append({
            "name": customer,
            "directory": path,
            "website_directory": public_path,
            "website_present": os.path.isfile(
                os.path.join(public_path, "index.html")
            ),
        })

    return customers


def get_storage_info():
    data_disk = run_readonly([
        "lsblk", "-dn", "-o", "NAME,SIZE", "/dev/sdb"
    ])

    logical_volumes = run_readonly([
        "lsblk", "-o", "NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS", "/dev/sdb"
    ])

    return {
        "data_disk": data_disk,
        "logical_volumes": logical_volumes,
    }


def get_backup_info():
    root = "/srv/backups/cloudguard"
    backups = []

    try:
        for name in sorted(os.listdir(root), reverse=True):
```

```
            path = os.path.join(root, name)

            if os.path.isdir(path):
                backups.append({
                    "name": name,
                    "created": datetime.fromtimestamp(
                        os.path.getmtime(path)
                    ).astimezone().isoformat(),
                })
    except (OSError, PermissionError):
        return {"available": False, "backups": []}

    return {
        "available": True,
        "count": len(backups),
        "backups": backups[:10],
    }


def get_service_info():
    return {
        service: service_status(service)
        for service in ("nginx", "ssh", "cron")
    }



@app.before_request
def require_dashboard_auth():
    username = os.environ.get("DASHBOARD_USERNAME")
    password_hash = os.environ.get("DASHBOARD_PASSWORD_HASH")

    if not username or not password_hash:
        return Response("Dashboard authentication is not configured.", 503)

    credentials = request.authorization

    if (
        credentials is None
        or not hmac.compare_digest(credentials.username, username)
        or not check_password_hash(password_hash, credentials.password)
    ):
```

```
        return Response(
            "Authentication required.",
            401,
            {"WWW-Authenticate": 'Basic realm="CloudGuard Admin"'}
        )


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/api/overview")
def overview():
    return jsonify({
        "system": get_system_info(),
        "disks": get_disk_info(),
        "customers": get_customer_info(),
        "storage": get_storage_info(),
        "backups": get_backup_info(),
        "services": get_service_info(),
    })


@app.route("/health")
def health():
    return jsonify({"status": "healthy", "application": "CloudGuard Dashboard"})


if __name__ == "__main__":
    app.run()
```

D.2 Frontend Template — templates/index.html (143 lines)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CloudGuard | Admin Dashboard</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <aside class="sidebar">
        <div class="brand">
            <div class="brand-icon">CG</div>
            <div>
                <h2>CloudGuard</h2>
                <span>HOSTING CONTROL</span>
            </div>
        </div>

        <div class="nav-label">OVERVIEW</div>
        <a class="nav-item active" href="#overview">▫   Dashboard</a>
        <a class="nav-item" href="#customers">▫   Customers</a>
        <a class="nav-item" href="#storage">▫   Storage</a>
        <a class="nav-item" href="#services">▫   Services</a>
        <a class="nav-item" href="#backups">▫   Backups</a>

        <div class="sidebar-bottom">
            <div class="server-indicator"></div>
            <div>
                <strong>CloudGuard Server</strong>
                <span>Private network</span>
            </div>
        </div>
    </aside>

    <main class="main-content">
        <header class="topbar">
            <div>
                <p class="eyebrow">CLOUD SERVICE PROVIDER</p>
                <h1>Dashboard</h1>
            </div>
            <div class="top-right">
                <span class="live-dot"></span>
```

```
                <span>Live monitoring</span>
                <span id="last-updated">Loading...</span>
            </div>
        </header>

        <section id="overview">
            <div class="section-heading">
                <div>
                    <h2>System overview</h2>
                    <p>Real-time health and performance</p>
                </div>
                <span class="server-tag" id="hostname">CloudGuard</span>
            </div>

            <div class="metrics-grid">
                <article class="metric-card">
                    <div class="metric-top"><span>CPU USAGE</span><span>▫</span></div>
                    <div class="metric-value" id="cpu">--%</div>
                    <div class="progress"><div id="cpu-bar"></div></div>
                    <p>Processor utilization</p>
                </article>

                <article class="metric-card">
                    <div class="metric-top"><span>MEMORY USAGE</span><span>▫</span></div
>
                    <div class="metric-value" id="memory">--%</div>
                    <div class="progress"><div id="memory-bar"></div></div>
                    <p id="memory-detail">RAM utilization</p>
                </article>

                <article class="metric-card">
                    <div class="metric-top"><span>SERVER UPTIME</span><span>▫</span></di
v>
                    <div class="metric-value" id="uptime">--</div>
                    <div class="metric-note">Since last boot</div>
                    <p>Continuous operation</p>
                </article>

                <article class="metric-card">
                    <div class="metric-top"><span>CUSTOMER SITES</span><span>▫</span></d
iv>
                    <div class="metric-value" id="site-count">--</div>
                    <div class="metric-note">Configured websites</div>
                    <p>Hosted customer websites</p>
                </article>
```

```
            </div>
        </section>

        <section class="content-grid" id="storage">
            <article class="panel">
                <div class="panel-heading">
                    <div><h2>Storage utilization</h2><p>Mounted filesystems</p></div>
                    <span class="panel-icon">▫</span>
                </div>
                <div id="disk-list" class="disk-list">Loading storage...</div>
            </article>

            <article class="panel" id="services">
                <div class="panel-heading">
                    <div><h2>Service health</h2><p>Critical server services</p></div>
                    <span class="panel-icon">▫</span>
                </div>
                <div id="service-list" class="service-list">Loading services...</div>
            </article>
        </section>

        <section class="content-grid lower-grid">
            <article class="panel" id="customers">
                <div class="panel-heading">
                    <div><h2>Customer hosting</h2><p>Website directory status</p></div>
                    <span class="panel-icon">▫</span>
                </div>
                <div id="customer-list">Loading customers...</div>
            </article>

            <article class="panel">
                <div class="panel-heading">
                    <div><h2>LVM storage</h2><p>Logical volume configuration</p></div>
                    <span class="panel-icon">▫</span>
                </div>
                <h3 class="subheading">Dedicated data disk</h3>
                <pre id="vg-info">Loading...</pre>
                <h3 class="subheading">Logical volumes and mounts</h3>
                <pre id="lv-info">Loading...</pre>
            </article>
        </section>
```

```
        <section class="panel backup-panel" id="backups">
            <div class="panel-heading">
                <div><h2>Backup history</h2><p>Latest CloudGuard backup snapshots</p></d
iv>
                <span class="panel-icon">▫</span>
            </div>
            <div id="backup-list">Loading backups...</div>
        </section>

        <footer>
            <span>CloudGuard Admin Dashboard</span>
            <span>Read-only monitoring · Auto-refresh every 15 seconds</span>
        </footer>
    </main>

    <script src="{{ url_for('static', filename='dashboard.js') }}"></script>
</body>
</html>
```

D.3 Stylesheet — static/style.css (186 lines)

```
:root {
    color-scheme: dark;
    --bg: #0b1020;
    --panel: #131c30;
    --panel-light: #19253c;
    --border: #26344d;
    --text: #e8eef8;
    --muted: #8b9ab3;
    --accent: #55d6be;
    --blue: #70a7ff;
    --red: #ff747e;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    background: var(--bg);
    color: var(--text);
    font-family: Inter, "Segoe UI", Arial, sans-serif;
}

.sidebar {
    position: fixed;
    inset: 0 auto 0 0;
    width: 245px;
    background: #0e1729;
    border-right: 1px solid var(--border);
    padding: 30px 20px;
    display: flex;
    flex-direction: column;
}

.brand { display: flex; align-items: center; gap: 12px; margin-bottom: 55px; }
.brand-icon {
    background: var(--accent);
    color: #07151a;
    font-weight: 900;
    border-radius: 10px;
    padding: 12px 9px;
}
.brand h2 { margin: 0; font-size: 19px; }
```

```
.brand span, .sidebar-bottom span {
    display: block;
    color: var(--muted);
    font-size: 10px;
    letter-spacing: 1.3px;
    margin-top: 5px;
}
.nav-label, .eyebrow {
    color: var(--muted);
    font-size: 10px;
    letter-spacing: 1.8px;
    font-weight: 700;
}
.nav-label { margin: 0 0 15px 10px; }
.nav-item {
    color: #aab8ce;
    text-decoration: none;
    padding: 13px 12px;
    margin-bottom: 6px;
    border-radius: 8px;
    font-size: 14px;
}
.nav-item:hover, .nav-item.active { background: #1b2b43; color: var(--accent); }

.sidebar-bottom {
    margin-top: auto;
    border-top: 1px solid var(--border);
    padding-top: 20px;
    display: flex;
    align-items: center;
    gap: 10px;
    font-size: 12px;
}
.server-indicator, .live-dot {
    width: 9px;
    height: 9px;
    border-radius: 50%;
    background: var(--accent);
    display: inline-block;
    box-shadow: 0 0 10px #55d6be77;
}
.sidebar-bottom span { letter-spacing: 0; font-size: 11px; }
```

```
.main-content { margin-left: 245px; padding: 35px 42px; max-width: 1600px; }
.topbar, .section-heading, .panel-heading, .top-right {
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.topbar { margin-bottom: 40px; }
.eyebrow { margin-bottom: 8px; }
h1 { font-size: 29px; margin: 0; }
.top-right { gap: 10px; color: var(--muted); font-size: 12px; flex-wrap: wrap; }
#last-updated { margin-left: 15px; }

.section-heading { margin-bottom: 18px; }
h2 { font-size: 17px; margin: 0 0 6px; }
.section-heading p, .panel-heading p { margin: 0; color: var(--muted); font-size: 12px;
}
.server-tag {
    border: 1px solid var(--border);
    padding: 8px 12px;
    border-radius: 7px;
    color: var(--accent);
    font-size: 11px;
}

.metrics-grid {
    display: grid;
    grid-template-columns: repeat(4, minmax(0, 1fr));
    gap: 17px;
    margin-bottom: 22px;
}
.metric-card, .panel {
    background: var(--panel);
    border: 1px solid var(--border);
    border-radius: 12px;
}
.metric-card { padding: 20px; min-width: 0; }
.metric-top { display: flex; justify-content: space-between; color: var(--muted); font-s
ize: 10px; letter-spacing: 1px; }
.metric-top span:last-child { color: var(--accent); font-size: 17px; }
.metric-value { font-size: 30px; font-weight: 700; margin: 20px 0 13px; }
.metric-card p { color: var(--muted); font-size: 11px; margin: 12px 0 0; }
.metric-note { color: var(--accent); font-size: 11px; min-height: 13px; }
```

```
.progress { height: 5px; background: #27364e; border-radius: 8px; overflow: hidden; }
.progress div { height: 100%; width: 0; background: var(--accent); border-radius: 8px; t
ransition: width .3s; }

.content-grid { display: grid; grid-template-columns: 1.2fr 1fr; gap: 18px; margin-botto
m: 18px; }
.panel { padding: 22px; min-width: 0; }
.panel-heading { margin-bottom: 22px; }
.panel-icon { color: var(--accent); font-size: 20px; }
.disk-row { margin: 17px 0; }
.disk-meta { display: flex; justify-content: space-between; font-size: 12px; margin-bott
om: 8px; gap: 12px; }
.disk-meta span:last-child { color: var(--muted); }
.disk-track { height: 7px; background: #27364e; border-radius: 8px; overflow: hidden; }
.disk-fill { height: 100%; background: var(--blue); border-radius: 8px; }
.disk-error { color: var(--red); font-size: 12px; }

.service-row, .customer-row, .backup-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 13px 0;
    border-bottom: 1px solid var(--border);
    font-size: 12px;
    gap: 12px;
}
.service-row:last-child, .customer-row:last-child, .backup-row:last-child { border-botto
m: 0; }
.status { font-size: 10px; padding: 5px 9px; border-radius: 20px; background: #173d37; c
olor: var(--accent); }
.status.bad { background: #49242d; color: var(--red); }
.customer-detail, .backup-detail { display: block; color: var(--muted); font-size: 10px;
 margin-top: 5px; }
.subheading { font-size: 11px; color: var(--muted); margin: 18px 0 8px; }
pre {
    white-space: pre-wrap;
    overflow-wrap: anywhere;
    background: #0e1729;
    border: 1px solid var(--border);
    border-radius: 7px;
    padding: 12px;
    color: #b8c8df;
    font-size: 11px;
    min-height: 35px;
}
.backup-panel { margin-top: 18px; }
footer { display: flex; justify-content: space-between; gap: 12px; color: var(--muted);
font-size: 10px; padding: 25px 3px; }
```

```
@media (max-width: 1050px) {
    .sidebar { width: 190px; }
    .main-content { margin-left: 190px; padding: 28px 24px; }
    .metrics-grid { grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 700px) {
    .sidebar { position: static; width: auto; padding: 17px; }
    .brand { margin-bottom: 20px; }
    .nav-label, .sidebar-bottom { display: none; }
    .nav-item { display: inline-block; padding: 9px; margin: 0; font-size: 11px; }
    .main-content { margin: 0; padding: 22px 15px; }
    .content-grid { grid-template-columns: 1fr; }
    .metrics-grid { gap: 10px; }
    .metric-card { padding: 14px; }
    .metric-value { font-size: 24px; }
    .topbar { align-items: flex-start; gap: 12px; flex-direction: column; }
    footer { flex-direction: column; }
}
```

D.4 Client-Side JavaScript — static/dashboard.js (104 lines)

```
const byId = (id) => document.getElementById(id);

function setProgress(id, value) {
    const safeValue = Math.max(0, Math.min(100, Number(value) || 0));
    byId(id).style.width = `${safeValue}%`;
}

function renderDisks(disks) {
    byId("disk-list").innerHTML = disks.map(disk => {
        if (disk.error) {
            return `<div class="disk-row disk-error">${disk.mount}: ${disk.error}</div>`
;
        }

        return `
            <div class="disk-row">
                <div class="disk-meta">
                    <strong>${disk.mount}</strong>
                    <span>${disk.used_gb} / ${disk.total_gb} GB · ${disk.percent}%</span
>
                </div>
                <div class="disk-track">
                    <div class="disk-fill" style="width:${Math.min(100, disk.percent)}%"
></div>
                </div>
            </div>`;
    }).join("");
}

function renderServices(services) {
    byId("service-list").innerHTML = Object.entries(services).map(([name, status]) => `
        <div class="service-row">
            <strong>${name}</strong>
            <span class="status ${status === "Running" ? "" : "bad"}">${status}</span>
        </div>`).join("");
}

function renderCustomers(customers) {
    byId("customer-list").innerHTML = customers.map(customer => `
        <div class="customer-row">
            <div>
                <strong>${customer.name}</strong>
                <span class="customer-detail">${customer.directory}</span>
            </div>
            <span class="status ${customer.website_present ? "" : "bad"}">
```

```
                ${customer.website_present ? "Website ready" : "Missing index"}
            </span>
        </div>`).join("");
}

function renderBackups(data) {
    if (!data.available) {
        byId("backup-list").textContent = "Backup directory is not accessible.";
        return;
    }

    if (!data.backups.length) {
        byId("backup-list").textContent = "No backups found.";
        return;
    }

    byId("backup-list").innerHTML = data.backups.map(backup => `
        <div class="backup-row">
            <strong>${backup.name}</strong>
            <span class="backup-detail">${new Date(backup.created).toLocaleString()}</sp
an>
        </div>`).join("");
}

async function refreshDashboard() {
    try {
        const response = await fetch("/api/overview", { cache: "no-store" });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);

        const data = await response.json();
        const system = data.system;

        byId("hostname").textContent = system.hostname;
        byId("cpu").textContent = `${system.cpu_percent}%`;
        setProgress("cpu-bar", system.cpu_percent);

        byId("memory").textContent = `${system.memory.percent}%`;
        byId("memory-detail").textContent =
            `${system.memory.used_gb} / ${system.memory.total_gb} GB RAM`;
        setProgress("memory-bar", system.memory.percent);

        byId("uptime").textContent = `${system.uptime_hours} hrs`;
        byId("site-count").textContent =
```

```
            data.customers.filter(c => c.website_present).length;

        byId("last-updated").textContent =
            `Updated ${new Date(system.timestamp).toLocaleTimeString()}`;

        renderDisks(data.disks);
        renderServices(data.services);
        renderCustomers(data.customers);
        renderBackups(data.backups);

        byId("vg-info").textContent = data.storage.data_disk;
        byId("lv-info").textContent = data.storage.logical_volumes;
    } catch (error) {
        byId("last-updated").textContent = "Connection error";
        console.error("CloudGuard dashboard:", error);
    }
}

refreshDashboard();
setInterval(refreshDashboard, 15000);
```

D.5 Python Dependencies — requirements.txt

```
Flask>=3.1,<4
gunicorn>=23,<24
psutil>=7,<8
```

Appendix E: System Configuration Manifests and Evidence Logs

E.1 Host SSH Configuration (~/.ssh/config)

```
Host cloudguard
    HostName 192.168.56.103
    User cloudadmin
    IdentityFile ~/.ssh/cloudguard_ed25519
    IdentitiesOnly yes
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

E.2 Host /etc/hosts Entries

```
# CloudGuard local demonstration hostnames
192.168.56.103    client1.cloudguard.local
192.168.56.103    client2.cloudguard.local
```

E.3 Nginx Client 1 Virtual Host Configuration

```
server {
    listen 80;
    listen [::]:80;
    server_name client1.cloudguard.local;
    root /srv/client1/public;
    index index.html;

    access_log /var/log/nginx/client1_access.log;
    error_log  /var/log/nginx/client1_error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

E.4 Nginx Client 2 Virtual Host Configuration

```
server {
    listen 80;
    listen [::]:80;
    server_name client2.cloudguard.local;
    root /srv/client2/public;
    index index.html;

    access_log /var/log/nginx/client2_access.log;
    error_log  /var/log/nginx/client2_error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

E.5 Key Service and Port Summary

Table E.1: CloudGuard service, port, and access summary

Service	Port / Schedule	Interface	Access / Purpose
OpenSSH (sshd)	TCP 22	enp0s8 (192.168.56.103)	Host-only network; Ed25519 key-based administration
Nginx (HTTP)	TCP 80	enp0s8 (192.168.56.103)	Host-only network; two customer virtual-hosted websites
CloudGuard Dashboard	TCP 5000	enp0s8 (192.168.56.103)	Host-only network; HTTP Basic Auth dashboard
Monitoring (cron)	Hourly (0 * * * *)	Local	Root-executed health monitoring script
Backup (cron)	Daily (0 2 * * *)	Local	Root-executed backup with 7-gen retention